create a prompt for a developpment project: Wildcard management studio. ask question to prepare it properly. it should include a graphical web user interface connected to a database, all put in a docker. It should understand all wildcards comfyui rules as provided here: ImpactWildcard.md We should be able to import both txt and yaml files the system should be able to parse the files, cut each prompt by tags or sentences, put all part into the database. the project should be able to deliver the prompt, as well as create new one. the project should be able to simulate wildcard usage in prompt the project should be able to organise prompt by theme, by copyright, by characters, by action type. the project should be able to connect to a local LLM provided by Koboldcpp (LostRuins/koboldcpp) or an API (chatgpt, claude, gemini, ...) that has to be setup in a settings tabs, as well as theme (dark or light) and language. the project should be able to make difference between tag prompting and natural language prompting. the project should know the specificities of differents diffusion models prompting such as: - SDXL prompting, here some references: https://civitai.com/articles/11432/ultimate-guide-to-creating-realistic-sdxl-prompts , https://blog.pixai.art/en/general-guide-of-writing-prompts-for-sdxl-models/ - Illustrious / Noobai

prompting, here some references: https://civitai.com/articles/23210/arctenoxs-simple-prompt-guide-for-illustrious-for-comfyuisite https://whatlab.ai/guides/illustrious-prompting-guide https://huggingface.co/Laxhar/noobai-XL-1.1/blob/main/README.md https://betterwaifu.com/blog/noobai-guide - Anima prompting, here some references: https://huggingface.co/circlestone-labs/Anima https://moescape.ai/posts/1-2-how-to-prompt-the-anima-ai-model-a-complete-guide https://moescape.ai/posts/84d36981-dea4-4365-b9e8-8b31d56b2a75 https://civitai.red/articles/30800/anima-prompting-guide-goodbye-stable-diffusion-crutches - Ideogram 4 prompting, here some references: https://ideogram.ai/blog/claude-mcp/ https://www.imagine.art/blogs/ideogram-4-0-prompt-guide the project should integrate this tool: KohakuBlueleaf/z-tipo-extensionhttps://huggingface.co/KBlueLeaf/DanTagGen-beta locally. Now with all those info, think of some more things the project could do, propose, and ask question to elaborate the final project prompt.

This is already a strong foundation. Before writing the final development prompt, I'd expand the scope so the resulting application becomes a complete **Wildcard Prompt Management Studio** instead of just a wildcard editor.

I would first propose additional features that naturally fit your vision, then ask questions that will determine the architecture.

---

**Additional features worth adding**

**1. Visual Prompt Builder**

Instead of editing text only.

A graphical interface where users can:

- drag & drop wildcard blocks
- reorder sections
- collapse categories
- preview the final prompt
- highlight wildcard depth
- see recursion visually

Example

Character

├── hair

├── clothes

├── pose

Environment

├── city

├── weather

Style

├── artist

├── quality

---

**2. Wildcard Dependency Graph**

Display

main prompt

|

+--- __character__

|

+--- __hair__

|

+--- __eyes__

Detect

- circular references
- missing files
- orphan wildcards
- duplicated definitions

---

**3. Wildcard Validator**

Checks

✓ missing wildcard

✓ infinite recursion

✓ invalid syntax

✓ unsupported syntax

✓ unused wildcard

✓ duplicate entries

✓ invalid weights

---

**4. Prompt Versioning**

Every edit becomes a revision.

Like Git.

v1

v2

v3

compare

restore

---

**5. Prompt Diff Viewer**

Highlight

Added

Removed

Modified

Very useful after LLM optimization.

---

**6. Prompt Library**

Manage prompts like assets.

Metadata

- title
- author
- copyright
- license
- source
- model compatibility

- creation date
- tags
- categories

---

**7. Image Association**

Attach generated images.

Each prompt can have

- generated image
- seed
- sampler
- CFG
- steps
- scheduler
- model
- LoRAs
- ControlNet

Making it a prompt database.

---

**8. Reverse Prompt Search**

Given

an image description

or prompt

Find

similar prompts already stored.

Semantic search using embeddings.

---

**9. AI Assistant**

Instead of simply calling ChatGPT.

Have assistants.

Examples

- Improve Prompt

- Convert to SDXL
- Convert to Illustrious
- Convert to Anima
- Convert to Danbooru tags
- Convert tags → natural language
- Compress prompt
- Expand prompt
- Remove repetition
- Explain prompt
- Detect conflicts

---

**10. Prompt Analyzer**

Statistics

Prompt length

Number of tags

Natural language %

Wildcard %

Artist %

Character %

Negative %

Quality %

Token estimation

Complexity score

---

**11. Prompt Templates**

Create reusable templates.

Example

Portrait

Landscape

Character sheet

Action scene

NSFW

Comic panel

Product photo

Fantasy illustration

---

**12. Wildcard Simulator**

Even more advanced.

Generate

100

1000

10000

samples.

Display

distribution

duplicates

frequency

probability

Useful for testing datasets.

---

**13. Randomizer Control**

Support

fixed seed

random seed

weighted randomness

avoid repetition

exclude values

force values

lock wildcard

---

**14. Tag Ontology**

Understand relationships.

Example

girl

└── teenager

└── child

└── adult

or

weapon

sword

spear

axe

This enables intelligent searching.

---

**15. Dataset Cleaning**

Automatically

merge duplicates

normalize tags

remove useless commas

fix spacing

alphabetize

deduplicate

---

**16. Prompt Translation**

Translate prompts while preserving tags.

Japanese

Chinese

French

English

without corrupting Danbooru tags.

---

**17. Prompt Conversion**

Convert between

Natural Language

↓

Danbooru Tags

↓

SDXL

↓

Anima

↓

Illustrious

↓

Ideogram

↓

Flux

↓

Midjourney style

↓

Stable Cascade

---

**18. Model Profiles**

Each model has its own prompting rules.

Instead of generic prompting.

Example

SDXL

Prompt style

Negative prompt

Weights

Best practices

Illustrious

Tag order

Character priority

Style priority

Ideogram

Natural language

Typography

Layout

The application automatically adapts.

---

**19. Plugin System**

Allow future extensions.

Examples

- new diffusion models
- new prompt parsers
- new AI providers
- custom exporters

---

**20. ComfyUI Integration**

Rather than only wildcard compatibility.

Direct connection.

Examples

- browse workflows
- inject prompts
- generate images
- retrieve metadata
- synchronize wildcards

---

**21. Automatic Wildcard Refactoring**

Convert

large TXT

↓

multiple wildcard files

Automatically.

---

**22. Full Text Search**

Search

contains

regex

fuzzy

semantic

wildcards

authors

characters

licenses

---

**23. Auto Tagger**

Using DanTagGen locally.

Generate tags from

- image
- description
- existing prompt

---

**24. Batch Processing**

Import

100 wildcard files

Normalize

Categorize

Merge

Export

Automatically.

---

**25. Export Formats**

Support

TXT

YAML

JSON

CSV

SQLite

Markdown

ComfyUI wildcard pack

---

**Questions before writing the final project prompt**

**Architecture**

1. Preferred backend?
    - Python (FastAPI)
    - Django
    - NodeJS
    - Rust
    - No preference
2. Preferred frontend?
    - React
    - Vue
    - Svelte
    - Angular
3. Database?
    - PostgreSQL
    - SQLite
    - MariaDB
    - MySQL
4. Docker Compose or Kubernetes-ready?

---

**AI**

5. Should local KoboldCpp have priority over cloud APIs?
6. Which providers should be supported initially?

- OpenAI
- Anthropic
- Gemini
- OpenRouter
- Ollama
- LM Studio
- KoboldCpp
- vLLM
- OpenAI-compatible APIs

7. Should multiple providers be selectable per task?

---

**Prompt intelligence**

8. Should the application automatically detect whether a prompt is:

- Danbooru
- SDXL
- Natural language
- Illustrious
- Anima
- Ideogram
- Mixed

---

**Parsing**

9. How aggressive should parsing be?

Only split by

,

.

newline

wildcards

or perform semantic parsing such as:

Subject

Style

Lighting

Camera

Pose

Action

Quality

Effects

Environment

using AI?

---

**Wildcards**

10. Should every wildcard become a database entity?

Example

Wildcard

Entries

Relationships

Usage statistics

References

instead of storing files only?

---

**Organization**

11. Should prompts support multiple simultaneous classifications?

Example

Fantasy

Elf

Female

Action

Forest

Night

Sword

Illustration

rather than a single category.

---

**Images**

12. Should images be stored inside the database or as files with metadata references?

---

**Search**

13. Should semantic search use embeddings locally?

Examples

- BGE
- E5
- Nomic
- Jina

---

**Security**

14. Single-user desktop application or multi-user server?

---

**Licensing**

15. Should prompt licenses be managed?

Example

Public Domain

CC0

CC-BY

Personal

Commercial

Unknown

---

**Prompt Generation**

16. Should the AI generate prompts by following a predefined grammar and structure, or be free-form?

---

**Diffusion Models**

17. Besides SDXL, Illustrious, NoobAI, Anima, and Ideogram 4, should the application include built-in profiles for additional models such as FLUX, Stable Diffusion 3.x, Pony, Hunyuan, or other emerging models?

---

**Database**

18. Should prompts be fully normalized?

For example:

Prompt

↓

Sections

↓

Sentences

↓

Tags

↓

Wildcards

↓

References

↓

Images

↓

Authors

instead of storing plain text?

---

**AI Memory**

19. Should the AI assistant learn from your own prompt library and use Retrieval-Augmented Generation (RAG) so it can propose prompts consistent with your existing style and wildcard collections?

---

**Performance**

20. Approximately how large do you expect the database to become?

- 10,000 prompts
- 100,000 prompts
- 1 million prompts
- More

This will influence indexing, caching, and search strategies.

---

**Ultimate Vision**

21. Is your goal simply to build a wildcard manager, or do you envision a **professional Prompt IDE** for diffusion models? A Prompt IDE would combine database management, visual editing, AI-assisted prompt engineering, wildcard simulation, model-specific optimization, ComfyUI integration, version control, semantic search, and local/cloud AI services into a unified development environment. This broader vision would guide the architecture toward extensibility and long-term maintainability.

A

Perfect. This keeps the project focused and technically clean.

**Deployment decision confirmed**

The application will be distributed as:

✅ **Docker Compose application only**

Target workflow:

git clone Wildcard-Management-Studio

cd Wildcard-Management-Studio

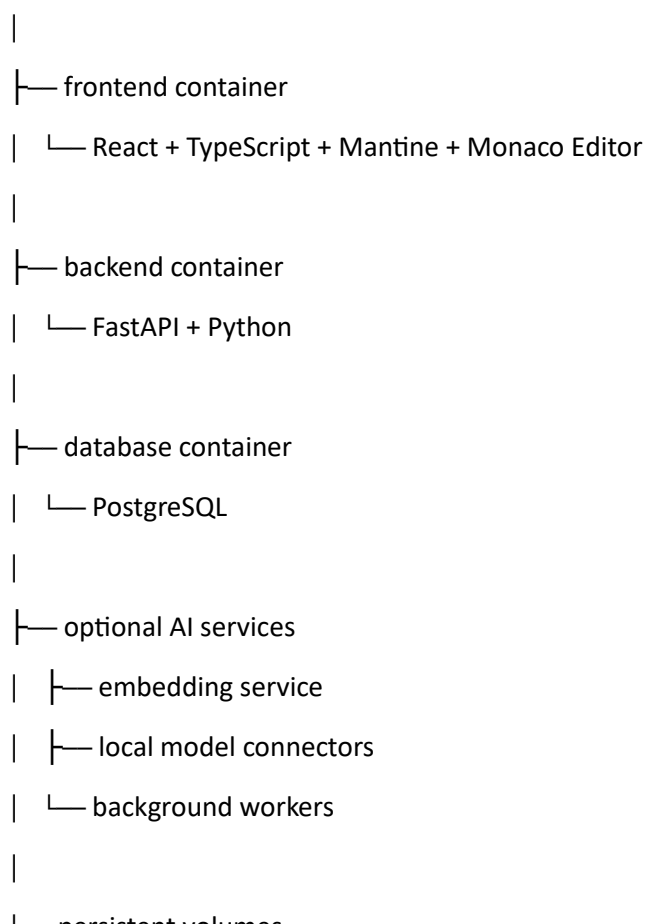
docker compose up -d

Then access:

http://localhost:<port>

The Docker stack will likely contain:

Wildcard Management Studio




The project prompt will specify:

- reproducible Docker builds
- environment variables

- persistent storage
- backup/export capability
- easy upgrade path

---

**Question 7 — File Import and Wildcard Storage**

You mentioned importing:

- .txt
- .yaml

and parsing ComfyUI wildcards.

Now we need to decide how the application should treat imported wildcard files.

**Option A — File-centric approach**

Keep the original files as the "source of truth".

Example:

wildcards/

├── character.txt

├── hair.yaml

└── clothing.txt

The database stores:

- file location
- metadata
- parsed content

Advantages:

- Closest to ComfyUI workflow
- Easy synchronization
- Easy export back

---

**Option B — Database-centric approach** ⭐

When importing:

character.txt

becomes:

Database entity:

Wildcard: character

Entries:

elf woman

cyberpunk soldier

warrior princess

The original file is only an import/export format.

Advantages:

- Powerful search
- Editing inside the IDE
- Statistics
- AI analysis
- Relationship mapping
- Versioning

---

**Option C — Hybrid ⭐ ⭐ (my recommendation)**

Keep both:

Original file

↓

Parser

↓

Database representation

↓

Editor modifications

↓

Export updated file

The system remembers:

- original source
- current database version
- changes
- synchronization status

This would be closest to how VS Code handles projects.

---

Which do you choose?

A, B, or C?

C

Excellent. This is the strongest choice for a Prompt IDE.

**Wildcard storage decision confirmed**

The system will use a **hybrid file + database architecture**.

The workflow becomes:

Import

|

v

TXT / YAML Wildcard Files

|

v

Parser Engine

|

+----------+

| |

v v

Database Original Files

(Model) (Source)

|

v

Visual Editor

|

v

Changes tracked

|

v

Export back to:

TXT / YAML / ComfyUI format

The application will therefore support:

**Source control features**

- Original file preservation
- Import history
- Modification tracking
- Diff comparison
- Restore previous versions
- Export modified wildcard packs

**Database features**

Each wildcard becomes a structured object:

Wildcard

├── Name

├── File source

├── Type

├── Entries

├── Tags

├── Themes

├── Characters

├── Actions

├── Copyright

├── Usage statistics

├── Dependencies

└── Version history

This fits perfectly with the Monaco IDE approach.

---

**Question 8 — Wildcard Syntax Compatibility**

You referenced the ComfyUI Impact Pack wildcard rules.

The application needs to know how strict compatibility should be.

Should the parser support:

**A. Only ComfyUI Impact Wildcard syntax**

Meaning:

- __wildcard_name__
- nested wildcards
- recursive expansion
- random selection
- file references
- YAML wildcard format according to the Impact Pack rules

Focus only on official compatibility.

---

**B. ComfyUI compatibility + extended syntax** ⭐

Everything in A, plus additional IDE features:

Examples:

__character|female__

(filter wildcard)

__style{realistic,anime}__

(condition)

__character*3__

(weight/repetition)

{option1|option2|option3}

(random inline choices)

[optional section]

(optional blocks)

---

**C. Build a full wildcard scripting language**

A new language inspired by:

- ComfyUI wildcards
- Stable Diffusion prompt syntax
- programming concepts

With:

- variables
- functions
- conditions
- loops
- templates

Example:

character = __female_character__

if(character.age > 20)

{

  add(__adult_clothing__)

}

Much more powerful, but much larger project.

---

Which direction do you want?

A, B, or C?

B

Excellent. This is a very good balance between **full ComfyUI compatibility** and making the application a true IDE rather than just a file browser.

**Wildcard syntax decision confirmed**

The project will support:

✅ **Full ComfyUI Impact Pack wildcard compatibility**

plus:

✅ **Extended IDE wildcard features**

The parser engine will therefore have two layers:

Wildcard Engine

|

├── Compatibility Layer

│ ├── __wildcard__

│ ├── nested wildcards

│ ├── YAML/TXT parsing

│ ├── recursive expansion

│ └── ComfyUI behavior matching

│

└── IDE Extension Layer

 ├── filters

 ├── weights

 ├── inline choices

 ├── conditional blocks

 ├── variables

 ├── prompt templates

 └── simulation controls

The Monaco editor can provide:

- syntax highlighting
- autocomplete
- validation
- error markers
- wildcard navigation
- "go to definition"

Example:

A portrait of __character__, wearing __clothing__,

in __environment__, with __style__

Clicking __character__ opens:

wildcards/

└── character.yaml

 ├── elf warrior

 ├── cyberpunk hacker

 └── medieval knight

---

**Question 9 — Prompt Generation and AI Assistance**

The project will integrate:

- KoboldCpp local LLM
- OpenAI
- Claude
- Gemini
- other APIs

Now we need to define **how much authority the AI has**.

**A. AI as assistant only**

The AI can:

- suggest prompts
- explain prompts
- improve wording
- convert models

But the user manually accepts every change.

---

**B. AI as interactive editor ⭐ (recommended)**

The AI can:

- modify selected text
- rewrite sections
- optimize prompts
- generate wildcard entries
- categorize prompts
- propose database organization

Example:

Select:

beautiful woman in forest

Ask:

"Convert for Illustrious"

AI returns:

1girl, forest, detailed eyes, long hair, ...

User accepts/rejects.

---

**C. AI autonomous optimization mode**

The AI can:

- analyze your library
- reorganize prompts
- merge duplicates
- create wildcard packs
- generate missing metadata
- improve the database automatically

Example:

"Optimize my fantasy character library"

The AI creates:

Fantasy/

├── races/

├── armor/

├── weapons/

├── poses/

└── environments/

---

Which level do you want?

A, B, or C?

(You can also answer something like "B + limited C".)

B

Excellent. This keeps the AI powerful while maintaining user control.

**AI role decision confirmed**

The application will use the AI as an **interactive prompt engineering assistant**.

The AI will not silently modify the database or files. Instead, it works like an advanced coding assistant inside the IDE.

---

**AI Assistant capabilities**

The AI panel will support:

**Prompt editing**

User selects:

beautiful woman in a forest

Commands:

- Improve prompt
- Expand details
- Make more cinematic
- Make more realistic
- Convert style
- Shorten prompt
- Remove redundancy

---

**Model conversion**

Examples:

SDXL prompt

↓

Illustrious prompt

↓

Anima prompt

↓

Ideogram 4 prompt

The AI uses the selected model profile.

---

**Wildcard generation**

Example:

User:

Create a clothing wildcard for fantasy female characters

AI generates:

fantasy_clothing:

- elven silk dress
- enchanted armor
- medieval leather outfit
- royal mage robes

User can:

- preview
- edit
- save

---

**Prompt explanation**

The AI can explain:

- why tags are ordered that way
- what each section does
- model-specific behavior
- potential conflicts

---

**Interactive workflow**

Similar to VS Code Copilot:

Monaco Editor

|

|

AI Assistant Panel

|

|

Accept / Reject / Modify

---

**Question 10 — AI Knowledge Source**

For model-specific prompting, the system needs knowledge about:

- SDXL
- Illustrious
- NoobAI
- Anima
- Ideogram 4
- future models

How should this knowledge be managed?

**A. Built-in knowledge only**

The application ships with curated guides and rules.

Advantages:

- predictable
- offline
- no setup

Disadvantage:

- harder to update

---

**B. RAG knowledge base ⭐ (recommended)**

The application stores documentation locally.

Example:

Knowledge Base

SDXL

├── prompting guide

├── examples

└── best practices

Illustrious

├── tag order

├── character rules

└── style rules

Anima

└── ...

The AI searches this before answering.

Advantages:

- expandable
- user can add guides
- works with local KoboldCpp
- keeps knowledge up to date

---

**C. Live web retrieval**

The AI searches online documentation when needed.

Advantages:

- always updated

Disadvantages:

- requires internet
- external dependency
- less reproducible

---

**D. Hybrid**

Local RAG first, optional web lookup second.

---

Which approach do you want? (A, B, C, or D)

B

Excellent. This choice fits the whole philosophy of the project: **a local, professional, extensible Prompt IDE**.

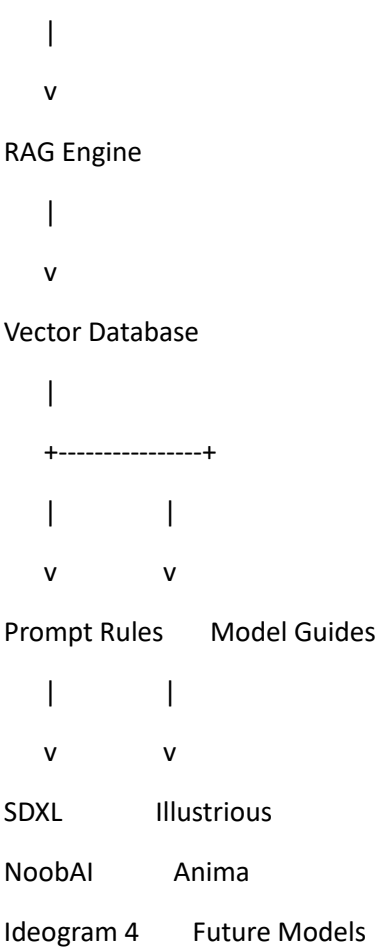
**AI knowledge decision confirmed**

The application will use a **local RAG (Retrieval-Augmented Generation) knowledge system**.

The AI assistant will not rely only on its general knowledge. It will query a dedicated knowledge base containing model-specific documentation.

---

**Knowledge Base Architecture**




---

**Knowledge sources**

The system will support importing:

- Markdown files
- TXT files
- PDF documentation
- Web pages (optional import)
- Prompt examples
- User-created notes
- Model documentation

---

**Each model profile can contain:**

Model Profile

│

├── Name

├── Version

├── Prompting philosophy

├── Tag ordering rules

├── Weighting rules

├── Recommended structure

├── Negative prompt rules

├── Examples

├── Do / Don't rules

└── References

Example:

SDXL Profile

Structure:

Subject → Details → Environment → Lighting → Camera

Preferred:

Natural language

Avoid:

Excessive Danbooru tags

---

**AI workflow example**

User:

Convert this SDXL prompt to Illustrious

The AI:

1. Detects source model

2. Retrieves Illustrious rules
3. Analyzes prompt structure
4. Converts tags
5. Suggests changes
6. Displays a diff

Example:

- beautiful woman standing in forest

+ 1girl, long hair, detailed eyes, forest background,

fantasy clothing, masterpiece, high detail

---

**Question 11 — Vector Database Choice**

Since we now have RAG, we need a vector search system.

Options:

**A. PostgreSQL + pgvector ⭐ (recommended)**

Use your existing PostgreSQL database.

Advantages:

- one database only
- simpler Docker setup
- good enough for millions of embeddings
- easy backup

---

**B. Dedicated vector database**

Examples:

- Qdrant
- Chroma
- Milvus
- Weaviate

Advantages:

- specialized
- very scalable

Disadvantages:

- another container
- more complexity

---

**C. Local embedded vector database**

Examples:

- FAISS
- SQLite vector extensions

Advantages:

- very lightweight

Disadvantages:

- less powerful

---

Which one do you prefer?

A, B, or C?

A

Excellent. This keeps the architecture elegant and avoids unnecessary complexity.

**Vector database decision confirmed**

The project will use:

✅ **PostgreSQL + pgvector**

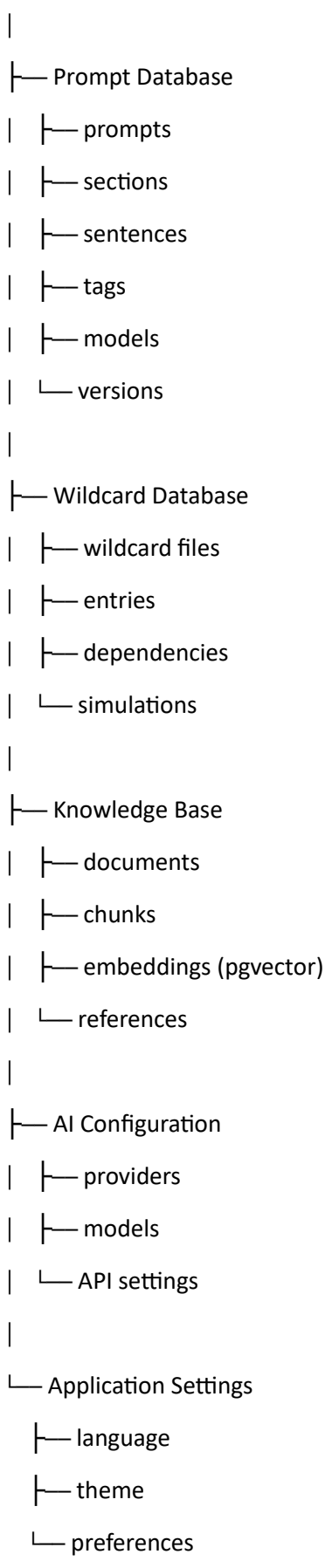
The same database will handle:

- structured prompt data
- wildcard data
- metadata
- version history
- AI embeddings
- RAG knowledge retrieval

---

**Final database architecture direction**

PostgreSQL


Prompt Database
prompts
sections
sentences
tags
models
versions
Wildcard Database
wildcard files
entries
dependencies
simulations
Knowledge Base
documents
chunks
embeddings (pgvector)
references
AI Configuration
providers
models
API settings
Application Settings
language
theme
preferences

This means:

- one backup contains everything
- simpler Docker deployment
- easy migration
- easier development

---

**Question 12 — Prompt Editor Experience**

Since you want a Monaco-style IDE, we need to decide how the main workspace should behave.

**A. Code-editor-first ⭐ (VS Code style)**

The central area is the Monaco editor.

Example:

```
+-----------------------------------------------+
| File tabs                                     |
+-----------------------------------------------+
|                                               |
| prompt.txt                                    |
|                                               |
| masterpiece, __character__, __style__         |
|                                               |
+-----------------------------------------------+
| AI Assistant | Database | Preview             |
+-----------------------------------------------+
```

Features:

- keyboard-driven
- fast editing
- familiar to developers
- ideal for advanced users

---

**B. Visual builder-first**

The main workspace is graphical.

Example:

Character

[female] [elf] [warrior]

Style

[anime] [cinematic]

Environment

[forest]

Features:

- easier for beginners
- drag/drop
- less text focused

---

**C. Hybrid IDE ⭐ ⭐ (my recommendation)**

Like modern design tools:

Main editor:

Monaco Editor

with optional panels:

LEFT:

Project explorer

Wildcard browser

CENTER:

Prompt editor

RIGHT:

AI assistant

Model analysis

Wildcard preview

BOTTOM:

Simulation / generation history

Users can switch between:

- raw prompt mode
- structured prompt mode
- visual tag mode
- wildcard graph mode

---

Which interface philosophy do you want?

A, B, or C?

C

Excellent. This is the choice that best matches the **Prompt IDE** concept.

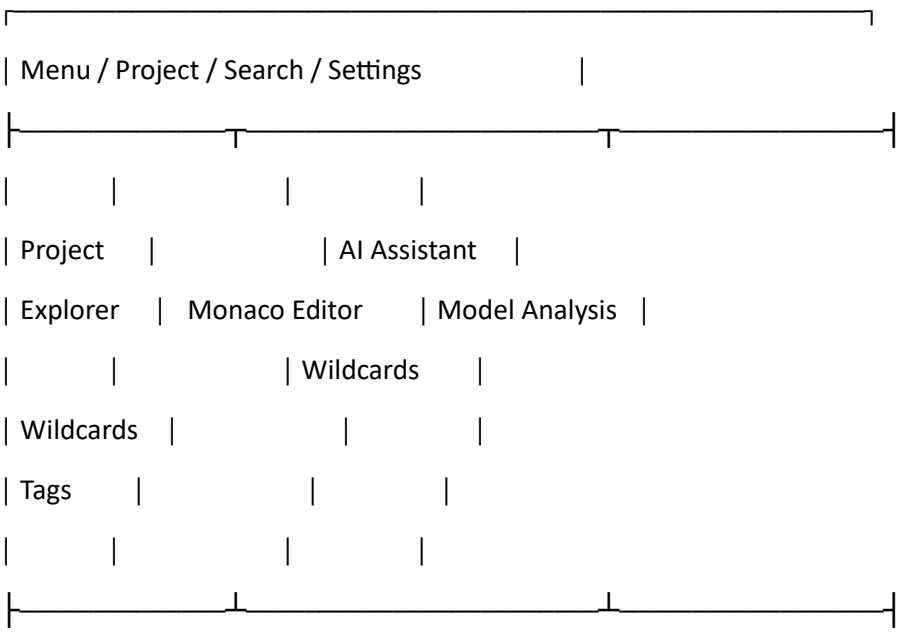
**Editor experience decision confirmed**

The application will use a **hybrid IDE interface** inspired by tools like VS Code, Unreal Engine, and modern creative software.

The workspace will be modular and panel-based.

---

**Main IDE Layout**



| Prompt Preview | Wildcard Simulation | History | Logs |

└──────────────────────────────────────────────────────────┘

---

**Workspace modes**

The user can switch between:

**1. Raw Prompt Mode**

Direct editing:

masterpiece, __character__, cinematic lighting

---

**2. Structured Prompt Mode**

Visual representation:

Subject

└── Character

Style

└── Cinematic

Lighting

└── Golden hour

---

**3. Tag Mode**

Danbooru/tag-oriented editing:

1girl

long hair

blue eyes

armor

forest

---

**4. Wildcard Graph Mode**

Visual dependency tree:

__character__

|

+-- __hair__

|

+-- __clothing__

---

**5. Model Optimization Mode**

Example:

Current:

SDXL prompt

Target:

Illustrious

The AI proposes conversion.

---

**Monaco IDE features to include**

- syntax highlighting
- autocomplete
- wildcard navigation
- tag suggestions
- prompt templates
- linting
- error markers
- search everywhere
- multi-file editing
- split views
- diff viewer
- version comparison

---

**Question 13 — Image Generation Integration**

The project will manage prompts, but should it also connect directly to image generators?

You already mentioned ComfyUI wildcard compatibility.

Which level of integration do you want?

**A. Prompt management only**

The application prepares prompts.

User copies them into ComfyUI manually.

---

**B. ComfyUI connection** ⭐

The application connects to a local ComfyUI instance.

Features:

- send prompts
- send negative prompts
- select workflows
- receive generated images
- retrieve metadata
- store seeds/settings
- link images back to prompts

---

**C. Full generation hub**

The application becomes a central interface for:

- ComfyUI
- Automatic1111
- Forge
- API-based generators
- local models

Like a universal image generation dashboard.

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the project focused while providing a very powerful workflow.

**Image generation integration decision confirmed**

The application will include:

✅ **Native ComfyUI integration**

but it will remain a **Prompt IDE first**, not a replacement for ComfyUI.

---

**ComfyUI Integration Architecture**

Wildcard Management Studio

|

|

v

ComfyUI Connector

|

|

v

Local ComfyUI API

|

|

v

Generated Images

|

|

v

Prompt Database + Metadata

---

**Supported features**

**Send prompts to ComfyUI**

The IDE can send:

- positive prompt
- negative prompt
- wildcard-expanded prompt

- seed
- model selection
- sampler
- scheduler
- CFG
- steps

---

**Workflow management**

The application can:

- browse available workflows
- select workflow templates
- associate workflows with models
- remember preferred workflows

Example:

SDXL Portrait Workflow

Illustrious Character Workflow

Anima Illustration Workflow

---

**Generation history**

Every generated image can store:

Image

|

├── Prompt version

├── Wildcards used

├── Expanded prompt

├── Seed

├── Model

├── LoRAs

├── Workflow

├— Date

└— Notes

---

**Wildcard simulation before generation**

Before sending:

__character__

becomes:

elf warrior princess

User can:

- regenerate wildcard choices
- lock some values
- test variations

---

**Question 14 — Image Storage**

When storing generated images and references, how should the application handle them?

**A. Store only metadata**

Images stay where ComfyUI saves them.

Database stores:

- filename
- path
- prompt
- seed
- parameters

Advantages:

- lightweight
- no duplication

---

**B. Import images into the application library**

The app manages its own image archive.

Advantages:

- unified asset browser

- easier backup
- better search

Disadvantages:

- more disk usage

---

**C. Hybrid ⭐ (recommended)**

Keep original ComfyUI files, but allow importing selected images.

Example:

ComfyUI output folder

|

|

+---- referenced images

|

+---- imported favorites

|

v

Prompt Library

---

Which do you prefer?

A, B, or C?

c

Excellent. This is the most practical choice for a creative workflow.

**Image storage decision confirmed**

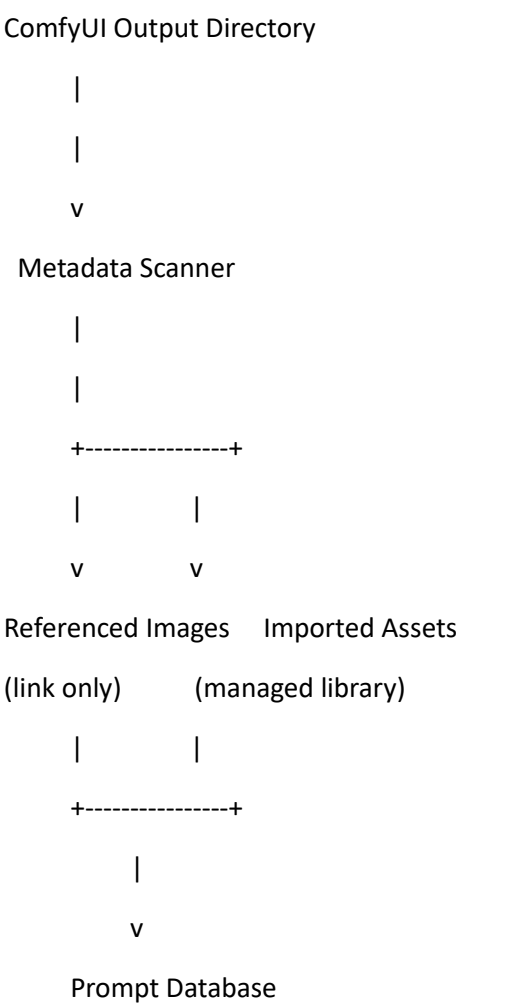
The application will use a **hybrid asset management system**.

The philosophy:

ComfyUI remains the generation engine. Wildcard Management Studio becomes the intelligence, organization, and knowledge layer.

---

**Image Asset Architecture**




---

**Supported image workflows**

**Automatic metadata retrieval**

When an image is detected:

Extract:

- prompt
- negative prompt
- seed
- sampler
- scheduler
- CFG
- steps
- model checkpoint

- LoRAs
- workflow JSON
- date created

---

**Image association**

Images can be linked to:

- prompts
- wildcard versions
- prompt revisions
- model profiles
- experiments

Example:

Prompt:

"Cyberpunk warrior"

Versions:

v1 → SDXL

v2 → Illustrious

v3 → Anima

Images:

img001.png

img002.png

img003.png

---

**Image library features**

- favorites
- rating
- notes
- tags

- search
- duplicate detection
- similarity search (future)

---

**Question 15 — Prompt Classification System**

You mentioned organizing prompts by:

- theme
- copyright
- characters
- action type

We need to define how classification works.

**A. Manual organization only**

User creates categories and tags.

Example:

Fantasy

└── Character

    └── Warrior

Simple but requires manual work.

---

**B. AI-assisted automatic classification** ⭐

When importing or creating prompts, the AI suggests:

Detected:

Theme:

✓ Fantasy

✓ Cyberpunk

Character:

✓ Female

✓ Warrior

Action:

✓ Fighting

Style:

✓ Anime

✓ Cinematic

User approves.

---

**C. Fully automatic semantic organization**

The AI continuously analyzes the entire library and creates:

- categories
- relationships
- clusters
- recommendations

Example:

Discovered collection:

"Dark Fantasy Female Characters"

contains 342 prompts

---

**D. Hybrid** ⭐ ⭐

Manual control + AI suggestions + semantic clustering.

---

Which one do you want?

A, B, C, or D?

D

Excellent. This is the most appropriate choice for a professional Prompt IDE.

**Prompt classification decision confirmed**

The system will use a **hybrid organization model**:

✅ Manual organization
✅ AI-assisted classification
✅ Semantic discovery and clustering

The user remains in control, but the system continuously helps organize a growing prompt library.

---

**Classification Architecture**

Prompt / Wildcard / Image

|

v

Analysis Pipeline

|

+----------+

| |

v v

Manual Tags AI Classification

| |

+----------+

|

v

Knowledge Graph

|

v

Search + Recommendations

---

**Classification dimensions**

The system will support multiple simultaneous classifications:

**Theme**

Examples:

Fantasy

Sci-Fi

Cyberpunk

Horror

Historical

Modern

Abstract

---

**Characters**

Examples:

Human

Elf

Robot

Monster

Animal

Original Character

Named Character

---

**Action Type**

Examples:

Portrait

Fighting

Walking

Running

Flying

Dancing

Sitting

Conversation

---

**Visual Style**

Examples:

Anime

Realistic

Cinematic

Painterly

Comic

Photography

Concept Art

---

**Copyright / Usage**

Examples:

Original

User Created

Public Domain

CC0

CC-BY

Unknown

Commercial Restricted

---

**AI suggestions example**

Import:

dark armored woman standing in ruined castle

AI proposes:

Theme:

✓ Dark Fantasy

Character:

✓ Female Warrior

Environment:

✓ Ruins

✓ Castle

Action:

✓ Standing Pose

Style:

✓ Cinematic Fantasy

Copyright:

Unknown

User can accept, modify, or reject.

---

**Question 16 — Tag Database**

Because the system needs to understand:

- Danbooru tags
- SDXL concepts
- Illustrious tags
- NoobAI tags
- model-specific terminology

How should the tag system work?

**A. Simple tag storage**

Just store strings:

1girl

long hair

blue eyes

armor

Fast and simple.

---

**B. Intelligent tag ontology** ⭐

Tags become structured concepts:

Example:

hair

|

+-- long hair

+-- short hair

+-- curly hair

character

|

+-- human

+-- elf

+-- android

The system understands relationships.

Benefits:

- better search
- duplicate detection
- AI suggestions
- model conversion

---

**C. External tag database integration**

Import existing databases:

- Danbooru tag database
- model-specific tag lists
- user collections

---

**D. Hybrid** ⭐ ⭐

Internal ontology + external databases + user-created tags.

---

Which one do you want?

A, B, C, or D?

D

Excellent. This is the right choice for a long-term Prompt IDE.

**Tag system decision confirmed**

The project will use a **hybrid intelligent tag ontology system**:

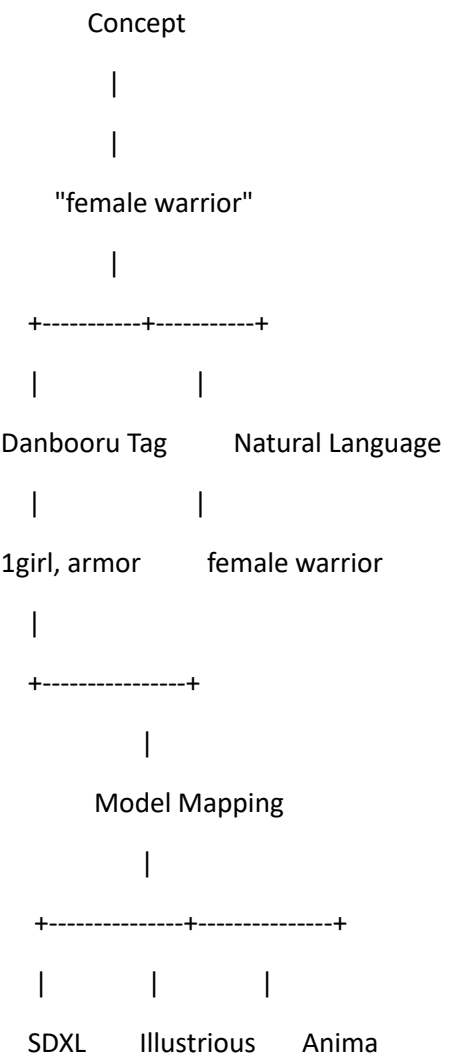
✅ Internal structured tag knowledge
✅ External tag database imports
✅ User-created custom tags
✅ Model-specific tag interpretation

This will allow the system to understand that the same concept can appear differently depending on the target model.

---

**Tag Knowledge Architecture**

Tag Knowledge System



---

**Tag entity structure**

A tag will contain:

Tag

|

├── Name

├── Aliases

├── Category

├── Parent tags

├── Child tags

├── Related concepts

├── Model compatibility

├── Popularity

├── Examples

└── User notes

---

**Supported tag categories**

Examples:

**Characters**

1girl

1boy

female

male

elf

robot

monster

---

**Appearance**

long hair

blue eyes

armor

dress

helmet

---

**Pose / Action**

standing

fighting

running

sitting

---

**Environment**

forest

castle

city

space

---

**Style**

anime

realistic

cinematic

oil painting

---

**External tag sources**

The project can import:

- Danbooru tag datasets
- user tag packs
- model documentation
- prompt examples
- custom dictionaries

---

**AI capabilities enabled**

The AI can now:

- suggest missing tags
- normalize duplicate tags
- convert natural language ↔ tags
- explain tags
- optimize ordering for a target model

Example:

Input:

A warrior woman in a forest

Possible outputs:

SDXL:

female warrior, forest, cinematic lighting

Illustrious:

1girl, armor, sword, forest background, detailed eyes

---

**Question 17 — z-tipo / DanTagGen Integration**

You requested integration of:

- KohakuBlueleaf z-tipo-extension
- KBlueLeaf/DanTagGen-beta locally

We need to decide how deep this integration should be.

**A. Simple tool integration**

The application can call DanTagGen:

Input:

- image

Output:

- tags

Then save the result.

---

**B. Full AI tagging pipeline** ⭐

DanTagGen becomes part of the Prompt IDE.

Workflow:

Image

|

v

DanTagGen

|

v

Generated Tags

|

v

Tag Normalizer

|

v

Ontology Matching

|

v

Database

|

v

Prompt Suggestions

Features:

- automatic tagging
- tag confidence scores
- duplicate cleanup
- model adaptation
- wildcard creation from tags

---

**C. Dataset creation system**

Everything in B plus:

- batch image processing
- dataset preparation
- caption generation
- training metadata creation

- LoRA dataset assistance

---

Which level do you want?

A, B, or C?

B

Excellent. This keeps the project aligned with the core mission: **a Prompt IDE**, while making DanTagGen a powerful internal intelligence component rather than turning the project into a full training dataset manager.

**z-tipo / DanTagGen integration decision confirmed**

The system will implement a **full AI tagging pipeline**.

DanTagGen will become one of the internal analysis engines of the application.

---

**AI Tagging Pipeline**

Image Asset

|

v

DanTagGen / z-tipo Engine

|

v

Raw Generated Tags

|

v

Tag Normalization Engine

|

v

Tag Ontology Matching

|

v

Prompt Database

|

v

AI Suggestions

---

**Features**

**Image → Tags**

Example:

Input image:

fantasy warrior woman in a forest

DanTagGen output:

1girl

long hair

armor

sword

forest

standing

---

**Tag cleanup**

The system detects:

Before:

long_hair

long hair

hair long

After:

long hair

---

**Tag enrichment**

The system can add:

long hair

parent:

hair

related:

flowing hair

wind effect

---

**Model adaptation**

The same image can generate different prompt representations:

SDXL:

female warrior in a mystical forest, cinematic lighting

Illustrious:

1girl, armor, sword, forest, detailed eyes

---

**Wildcard creation**

From tagged images:

Generate wildcard:

fantasy_warrior_character.yaml

Example:

fantasy_warrior_character:

- armored female knight
- elven warrior
- magical swordswoman

---

**Question 18 — Language Support**

You mentioned a language setting.

How broad should the localization system be?

**A. English only initially**

Simplest development.

---

**B. English + major languages**

Initial support:

- English
- French
- German
- Spanish
- Japanese
- Chinese

---

**C. Full localization framework**

The application is built from the beginning with translation files:

Example:

/locales

en.json

fr.json

ja.json

zh.json

de.json

Users can add translations later.

---

**D. AI-assisted translation system**

Everything in C plus:

- automatic translation
- user-editable translations
- community language packs

---

Which option do you want?

D

Excellent. This fits the philosophy of making the application a long-lived, extensible tool.

**Language support decision confirmed**

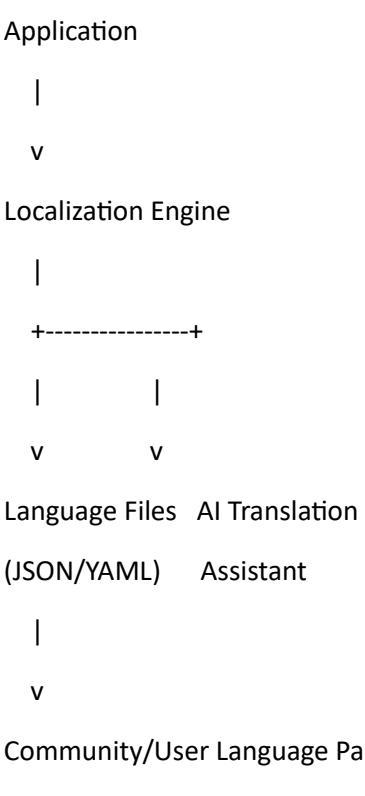
The application will use a **full localization framework with AI-assisted translation capabilities**.

The goal:

The interface, documentation, AI interactions, and prompt organization should be usable internationally without hard-coding a single language.

---

**Localization Architecture**




---

**Supported features**

**Interface localization**

All UI elements:

- menus
- buttons
- dialogs
- settings
- tooltips

- error messages

stored externally:

locales/

en.json

fr.json

ja.json

zh.json

de.json

---

**AI-assisted translation**

The AI can help translate:

- interface text
- documentation
- prompt descriptions
- wildcard metadata
- categories
- notes

while preserving technical terms.

Example:

Do not translate:

__character__

1girl

LoRA

ControlNet

CFG

but translate:

Fantasy warrior character

---

**Prompt language intelligence**

The system should understand:

- English prompts
- French prompts
- Japanese prompts
- Chinese prompts

while keeping model-specific tags intact.

Example:

Input:

femme guerrière dans une forêt magique

AI understands:

female warrior, magical forest

and can optimize for SDXL / Illustrious / Anima.

---

**Question 19 — Settings Management**

You mentioned a settings tab for:

- KoboldCpp
- ChatGPT
- Claude
- Gemini
- theme
- language

We need to decide how comprehensive the settings system should be.

**A. Basic settings**

Only:

- theme
- language
- API keys
- LLM endpoint

---

**B. Advanced developer settings** ⭐

Everything in A plus:

AI:

- provider priority
- model selection
- temperature
- context size
- max tokens
- embedding model

Database:

- backup
- maintenance
- indexing

Interface:

- panel layouts
- shortcuts
- editor behavior

---

**C. Professional IDE settings** ⭐ ⭐

Everything in B plus:

VS Code style configuration:

```
{
 "editor.fontSize": 14,
 "editor.wordWrap": true,
 "editor.minimap": true,
 "editor.theme": "dark"
}
```

Includes:

- keyboard shortcuts
- extensions/plugins
- workspace profiles
- custom themes
- custom commands

---

Which level do you want?

A, B, or C?

C

Excellent. This confirms the application should behave like a **real development environment**, not just a web dashboard.

**Settings system decision confirmed**

The project will implement a **professional IDE-style configuration system**.

The philosophy:

Users should be able to customize Wildcard Management Studio the same way developers customize VS Code.

---

**Settings Architecture**

Settings System

Global Settings

|

|

+----------------+

| |

v v

User Preferences Workspace Profiles

|

v

Configuration Engine

|

+----------------+

| |

v v

UI Settings AI Settings

---

**Configuration categories**

**Interface**

Theme:

├── Dark

├── Light

└── Custom

Language:

├── English

├── French

├── Japanese

└── User packs

Layout:

├── Panel positions

├── Sidebar visibility

└── Workspace presets

---

**Monaco Editor**

VS Code-style settings:

```
{
  "editor.fontSize": 14,
  "editor.wordWrap": true,
  "editor.minimap": true,
  "editor.autoComplete": true,
  "editor.syntaxHighlighting": true
}
```

Additional:

- prompt syntax colors

- wildcard colors
- tag colors
- bracket matching
- formatting rules

---

**AI Providers**

Example:

AI Providers

KoboldCpp
├── URL
├── Model name
├── Context size
└── Priority

OpenAI
├── API key
├── Model
└── Parameters

Claude
├── API key
└── Model

Gemini
├── API key
└── Model

---

**AI Behavior**

Settings:

- temperature

- creativity
- response length
- preferred model
- RAG usage
- automatic suggestions
- prompt style preference

---

**Workspace Profiles**

Example:

Anime Workflow Profile

Model:

Illustrious

Tag style:

Danbooru

Wildcard:

Enabled

AI Assistant:

KoboldCpp

Another:

Realistic Photography Profile

Model:

SDXL

Prompt style:

Natural language

---

**Extensions System Preparation**

The architecture will allow future plugins:

Examples:

- new model support
- new prompt formats
- new AI providers
- custom analyzers

---

**Question 20 — Project File Organization**

Since this is an IDE, we need to define how users organize their work.

Should the application use:

**A. Database only**

Everything lives in PostgreSQL.

Users do not see project files.

---

**B. Workspace folders** ⭐

Like VS Code.

Example:

My_AI_Project/

```
├── prompts/
│   ├── characters/
│   ├── environments/
│   └── styles/

├── wildcards/
│   ├── character.yaml
│   └── clothing.txt

├── images/
```

├── workflows/

└── project.json

The database indexes everything.

---

**C. Full IDE project system**

Everything in B plus:

- project files
- workspace settings
- versioning
- templates
- project sharing/export
- project snapshots

Example:

Fantasy_Project.wstudio

---

Which one do you want?

A, B, or C?

B

Excellent. This is a very good fit for the **Prompt IDE** concept while avoiding unnecessary complexity.

**Project organization decision confirmed**

The application will use a **workspace folder system**, similar to VS Code.

The database remains the intelligence layer, but users work with real files and folders.

---

**Workspace Architecture**

Workspace Folder

My_Prompt_Project/

```
├── prompts/
|   ├── characters/
|   ├── environments/
|   ├── styles/
|   └── complete_prompts/

├── wildcards/
|   ├── characters.yaml
|   ├── clothing.txt
|   └── locations/

├── tags/
|   └── custom_tags.yaml

├── models/
|   └── model_profiles/

├── workflows/
|   └── comfyui/

├── images/
|   └── references/

├── knowledge/
|   └── guides/

└── workspace.json
```

---

**Workspace features**

**File synchronization**

The system monitors:

- new files
- modified files
- deleted files
- renamed files

and updates the database index.

---

**Project explorer**

The left panel will behave like VS Code:

Explorer

📁 Fantasy Project

├─ 📄 elf_character.yaml

├─ 📄 dark_forest.prompt

├─ 📄 warrior_template.txt

└─ 📁 generated_images

---

**Workspace-specific settings**

Example:

```
{
  "defaultModel": "Illustrious",
  "promptStyle": "danbooru",
  "defaultWildcardPath": "./wildcards",
  "aiProvider": "koboldcpp"
}
```

---

**Import/export**

A complete workspace can be:

- backed up
- copied

- archived
- shared

without losing the database relationship.

---

**Question 21 — Prompt File Formats**

You mentioned TXT and YAML import.

Should the IDE support a dedicated native prompt format?

**A. Only existing formats**

Support:

- TXT
- YAML
- JSON
- Markdown

No custom format.

---

**B. Add a native .prompt format** ⭐

Example:

character.prompt

Containing:

name: Cyberpunk Warrior

model:
 - SDXL
 - Illustrious

prompt:
 character:
  - female warrior
  - cyber armor

style:

- cinematic

wildcards:

- __weapon__
- __city__

metadata:

copyright: original

Advantages:

- structured editing
- version control friendly
- model conversion
- AI analysis
- clean project files

---

**C. Full custom scripting format**

Create a complete prompt language.

Example:

PROMPT FantasyWarrior {

character:

__female_character__

style:

cinematic

model:

Illustrious

}

---

Which one do you want?

A, B, or C?

B but should be able to convert into yaml

Excellent. This is a strong choice because it gives the IDE a **native professional format** while keeping compatibility with existing tools and workflows.

**Prompt file format decision confirmed**

The project will implement:

✅ Native .prompt format as the primary IDE format
✅ Import/export compatibility with YAML
✅ Support for TXT, JSON, Markdown where useful

The .prompt format becomes the equivalent of a project source file, while YAML becomes an interchange format.

---

**Prompt File Architecture**

Example:

my_character.prompt

contains:

name: Cyberpunk Warrior

metadata:

author: user

copyright: original

themes:

- cyberpunk

- action

models:

compatible:

- SDXL

- Illustrious

prompt:

subject:
- female warrior

appearance:
- cyber armor
- glowing eyes

environment:
- futuristic city

style:
- cinematic lighting

wildcards:

- __character__
- __weapon__
- __city__

negative:

- blurry
- low quality

---

**Conversion system**

The IDE will support:

.prompt

|

+---- Export ----> YAML

|

+---- Export ----> TXT

|

+---- Export ----> JSON

|

+---- Export ----> ComfyUI prompt format

And reverse:

YAML

TXT

JSON

Markdown

|

v

Prompt Parser

|

v

.prompt

---

**Benefits**

The .prompt format enables:

- structured editing
- Monaco syntax highlighting
- autocomplete
- AI understanding
- version comparison

- model conversion
- wildcard references
- metadata preservation

---

**Question 22 — Prompt Model Profiles**

You listed support for:

- SDXL
- Illustrious
- NoobAI
- Anima
- Ideogram 4

How should model knowledge be handled?

**A. Fixed built-in profiles**

The application ships with:

- SDXL profile
- Illustrious profile
- NoobAI profile
- Anima profile
- Ideogram profile

Users cannot modify them.

---

**B. Editable model profiles** ⭐

Each model has a configurable profile:

SDXL Profile

Prompt style:

Natural language

Tag preference:

Medium

Order:

Subject → Details → Environment

Negative:

Recommended

Weighting:

Supported

Users can modify and create profiles.

---

**C. Community model marketplace**

Everything in B plus:

- import model profiles
- share profiles
- download community knowledge
- version profiles

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the system powerful and future-proof without adding unnecessary complexity.

**Model profile decision confirmed**

The application will use **editable model profiles**.

The Prompt IDE will not hard-code prompting rules. Instead, every diffusion model will be represented as a configurable knowledge profile.

---

**Model Profile Architecture**

Model Profile

Name:

SDXL

Version:

1.0

Prompting Style:

Natural Language

Tag Usage:

Medium

Prompt Structure:

Subject

→ Details

→ Environment

→ Lighting

→ Camera

→ Quality

Weighting:

Enabled

Negative Prompt:

Supported

Wildcard Behavior:

Compatible

AI Instructions:

Custom rules

---

**Initial built-in profiles**

The project will include profiles for:

**SDXL**

Knowledge:

- natural language prompting
- composition
- lighting
- realism structure
- quality modifiers

---

**Illustrious**

Knowledge:

- anime-oriented prompting
- Danbooru-style tags
- character priority
- tag ordering

---

**NoobAI**

Knowledge:

- anime tag conventions
- character emphasis
- style handling
- quality tags

---

**Anima**

Knowledge:

- natural language + artistic description
- cinematic composition
- concept-driven prompts

---

**Ideogram 4**

Knowledge:

- natural language emphasis
- typography handling
- image composition
- layout descriptions

---

**User customization**

Users can create:

My Custom Model

Base:

Illustrious

Modified:

- stronger character tags
- custom quality tags
- custom wildcard behavior

---

**AI behavior**

When optimizing a prompt:

The AI checks:

Selected Model Profile

+

RAG Knowledge Base

+

User Preferences

↓

Optimized Prompt

---

**Question 23 — Prompt Simulation**

You requested that the system should simulate wildcard usage.

How advanced should this simulator be?

**A. Basic expansion**

Example:

Input:

__character__ in __location__

Output:

elf warrior in ancient forest

---

**B. Advanced simulator** ⭐

Includes:

- multiple generations
- seed control
- locked wildcards
- probability display
- duplicate detection
- expansion history

Example:

Simulation 1000 samples

Character:

Elf warrior 32%

Cyber knight 25%

Mage 18%

Location:

Forest 40%

Castle 30%

City 30%

---

**C. Full prompt testing laboratory**

Everything in B plus:

- compare wildcard versions
- compare model outputs
- optimize wildcard balance
- detect boring/repeated combinations
- AI recommendations

---

Which one do you want?

A, B, or C?

C

Excellent. This is a defining feature for making the application a true **Prompt IDE** rather than a wildcard browser.

**Prompt simulation decision confirmed**

The project will include a **full Prompt Testing Laboratory**.

The wildcard engine becomes not only a generator, but also an analysis and optimization tool.

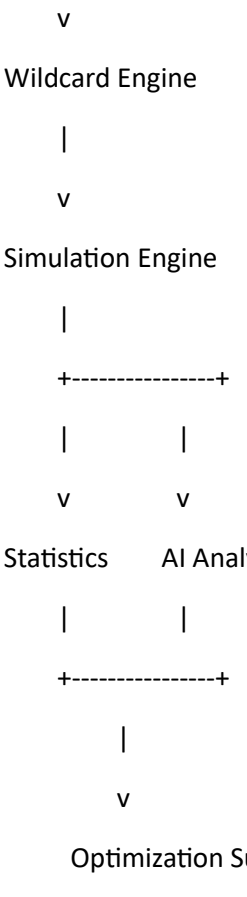
---

**Prompt Simulation Laboratory**

Architecture:

Prompt Template

|




---

**Simulation features**

**Wildcard expansion testing**

Example:

Input:

A portrait of __character__ wearing __clothing__ in __environment__

Run:

1000 simulations

Results:

Character distribution:

| | |
|---|---|
| Elf warrior | 28% |
| Cyberpunk hacker | 24% |
| Mage | 18% |
| Knight | 15% |
| Other | 15% |

---

**Controlled simulation**

Users can:

**Lock values**

__character__ = elf warrior

while testing:

__clothing__

__environment__

---

**Force exclusions**

Example:

Avoid:

duplicate characters

modern clothes

---

**Compare wildcard versions**

Example:

character_v1.yaml

vs

character_v2.yaml

Analysis:

Added:

+ 120 new entries

Removed:

- 30 duplicates

Balance:

Improved diversity +18%

---

**AI analysis**

The assistant can detect:

**Repetition problems**

Example:

"Your wildcard has 70% medieval armor entries. Consider adding casual clothing."

---

**Bias problems**

Example:

"This wildcard produces mostly male characters. Add more diversity entries."

---

**Weak combinations**

Example:

"Forest + spaceship occurs frequently but has low semantic coherence."

---

**Prompt quality scoring**

Possible metrics:

Prompt Score


---

**Question 24 — Search System**

The application will eventually contain:

- thousands of prompts
- wildcards
- tags
- images
- knowledge documents

How should searching work?

**A. Traditional search only**

Like a file manager:

- exact text
- filename
- tags
- filters

---

**B. Intelligent semantic search** ⭐

Using embeddings:

User searches:

"dark female fantasy warrior"

Finds:

- "shadow knight"
- "female paladin"
- "black armor character"

even if the words are different.

---

**C. Advanced AI search engine**

Everything in B plus:

- natural language queries
- similarity search
- image similarity
- prompt recommendations
- relationship discovery

Example:

User:

"Find prompts similar to this image"

The system searches:

- images
- prompts

- tags
- wildcards
- model profiles

---

**D. Hybrid** ⭐ ⭐

Traditional + semantic + AI search.

---

Which one do you want?

A, B, C, or D?

D

Excellent. This is the right choice for the scale of the project.

**Search system decision confirmed**

The application will implement a **hybrid AI-powered search engine**.

The goal:

Users should be able to find content the way they think, not only the way they typed it.

---

**Search Architecture**

Search Request

```
    |
    |
    +----------------+
    |                |
    v                v
Traditional Search   Semantic Search
(PostgreSQL)         (pgvector)
    |                |
    +----------------+
            |
```

v

AI Ranking Layer

|

v

Search Results

---

**Search capabilities**

**Traditional search**

Supports:

- filenames
- prompt text
- wildcard names
- tags
- metadata
- authors
- copyright
- model compatibility

Example:

"elf warrior"

Finds:

- prompts
- wildcard entries
- images
- notes

---

**Semantic search**

The system understands concepts.

Example:

Query:

dark cybernetic female fighter

Can find:

android assassin

neon armored woman

futuristic warrior girl

even without exact words.

---

**AI search assistant**

Natural language:

User:

"Find my best prompts for cinematic fantasy portraits using Illustrious"

The system filters:

Model:

Illustrious

Category:

Portrait

Theme:

Fantasy

Style:

Cinematic

Rating:

High

---

**Image similarity search**

Using image embeddings:

Input image:

↓

Find:

- similar generated images

- related prompts
- matching wildcards
- similar compositions

---

**Recommendation engine**

Example:

User opens:

Cyberpunk female character

Suggestions:

Similar prompts:

+ Neon city warrior

+ Android assassin

+ Futuristic armor

Related wildcards:

+ cyber_clothing

+ futuristic_weapons

---

**Question 25 — Version Control**

Since this is an IDE and prompts are structured assets, how should versioning work?

**A. Basic history**

Store:

- previous versions
- timestamps
- restore option

---

**B. Git-like version control** ⭐

Inspired by Git:

Features:

- commits
- diffs

- branches
- tags
- rollback

Example:

Prompt:

```
v1
 |
 +-- v2
     |
     +-- v3
```

---

**C. Full collaboration workflow**

Everything in B plus:

- Git repository integration
- pull/push
- remote backups
- team workflows

---

Which one do you want?

A, B, or C?

B

Excellent. This fits the IDE philosophy perfectly.

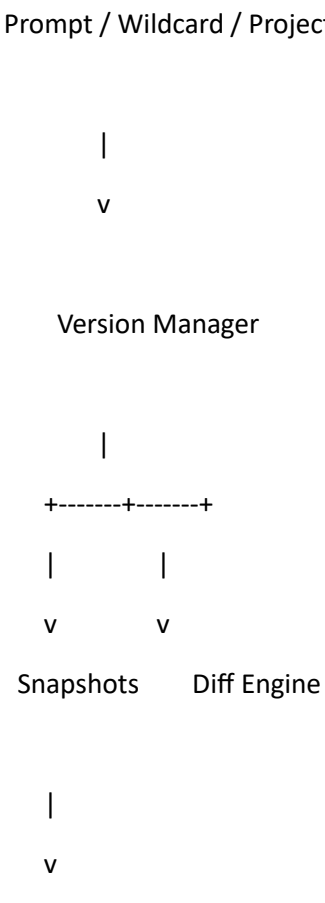
**Version control decision confirmed**

The project will include a **Git-inspired internal version control system**.

It will not require Git itself, but will provide the concepts needed for creative prompt development.

---

**Prompt Version Control Architecture**




---

**Version features**

**Automatic snapshots**

Create versions when:

- saving major changes
- importing updates
- AI modifications
- wildcard regeneration
- model conversion

---

**Commit system**

Example:

Commit:

"Converted SDXL prompt to Illustrious"

Changes:

+ added character tags

+ reordered style tags

- removed redundant quality tags

---

**Diff viewer**

Using Monaco Diff Editor:

id="4f3mzt"

- beautiful woman in forest

+ 1girl, long hair, forest,

+ cinematic lighting, detailed eyes

---

**Branching concepts**

Useful for experimentation:

Main Prompt

|

+---- SDXL Version

|

+---- Illustrious Version

|

+---- Experimental AI Version

---

**Restore**

Users can:

- restore old prompts
- compare versions

- duplicate versions
- merge improvements

---

**Wildcard versioning**

Also applies to:

- YAML files
- TXT files
- wildcard databases

Example:

character.yaml

v1:

100 entries

v2:

150 entries

v3:

removed duplicates

improved balance

---

**Question 26 — Backup and Export**

Since this application will contain:

- prompts
- images references
- wildcards
- AI knowledge
- settings
- databases

How should backup work?

**A. Manual export only**

User chooses when to export:

- JSON
- YAML
- ZIP

---

**B. Automatic local backups** ⭐

System creates:

- scheduled backups
- database snapshots
- workspace archives

---

**C. Professional backup system**

Everything in B plus:

- encrypted backups
- backup rotation
- restore points
- selective restore
- migration tools

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the application practical while protecting a valuable prompt library.

**Backup decision confirmed**

The project will implement an **automatic local backup system**.

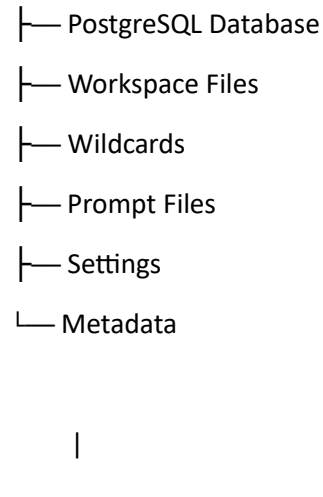
The objective:

A user's prompt knowledge base should never be lost because of an accidental edit, failed import, database corruption, or Docker issue.

---

**Backup Architecture**

Application Data




---

**Backup features**

**Automatic snapshots**

The system can create backups:

- daily
- weekly
- before major operations

Examples:

Before:

- database migration
- wildcard import

- bulk AI modification
- mass cleanup

---

**Backup contents**

A backup can include:

Backup Archive

├── database.dump
├── workspace/
├── wildcards/
├── prompts/
├── settings.json
└── metadata.json

---

**Restore options**

Users can restore:

- full workspace
- only prompts
- only wildcards
- only settings
- previous database state

---

**Docker consideration**

Persistent volumes will support:

- database backup
- workspace backup
- easy migration to another machine

---

**Question 27 — Extension / Plugin System**

Since this is becoming a long-term IDE, should the application support extensions?

Examples:

- new model support
- new import formats
- new AI providers
- new analyzers
- custom exporters
- custom UI panels

**A. No plugin system**

Everything is built into the core.

---

**B. Internal extension architecture** ⭐

The application is designed with modules:

extensions/

├── models/
├── importers/
├── exporters/
├── ai/
└── analyzers/

Developers can add features.

---

**C. Full marketplace-style plugin system**

Everything in B plus:

- install/uninstall plugins
- plugin repository
- version management
- community extensions

---

Which one do you want?

A, B, or C?

B

Excellent. This is the right balance for a serious open-ended tool.

**Extension architecture decision confirmed**

The project will include an **internal modular extension architecture**.

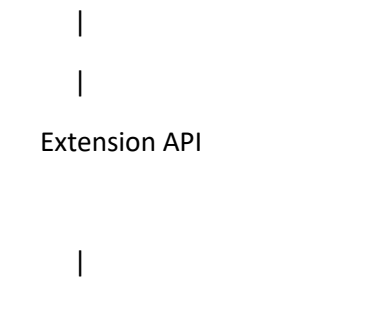
The goal:

Keep the core application stable while allowing new capabilities to be added without rewriting the system.

---

**Extension Architecture**

Wildcard Management Studio Core




---

**Extension categories**

**Model extensions**

Add support for:

- new diffusion models
- new prompt rules
- new tag systems

Example:

extensions/models/

├— flux/

├── stable_diffusion_3/

└── future_model/

---

**Import extensions**

Support:

- new file formats
- external databases
- prompt libraries

Examples:

TXT importer

YAML importer

CivitAI importer

Custom database importer

---

**Export extensions**

Examples:

- ComfyUI format
- Automatic1111 format
- Forge format
- API payload format

---

**AI extensions**

Examples:

- KoboldCpp connector
- OpenAI connector
- Claude connector
- Gemini connector
- local embedding models

---

**Analysis extensions**

Examples:

- prompt quality analyzer
- copyright detector
- duplicate detector
- tag optimizer

---

**Development benefits**

This allows:

- cleaner codebase
- easier testing
- community contributions later
- adding models without changing the core

---

**Question 28 — Security and API Key Management**

The application will store:

- OpenAI keys
- Claude keys
- Gemini keys
- KoboldCpp endpoints
- possibly other credentials

How should this be handled?

**A. Simple local storage**

Store settings in database/config files.

---

**B. Encrypted local storage** ⭐

Encrypt:

- API keys
- tokens
- sensitive configuration

Use:

- local encryption key
- environment variables
- secure storage

---

**C. Enterprise-grade secrets management**

Use:

- Vault
- OS keychain
- hardware-backed storage

More complex.

---

Which option do you want?

A, B, or C?

B

Excellent. This is the appropriate level for a **single-user local application**.

**Security decision confirmed**

The project will implement **encrypted local credential storage**.

The application will protect:

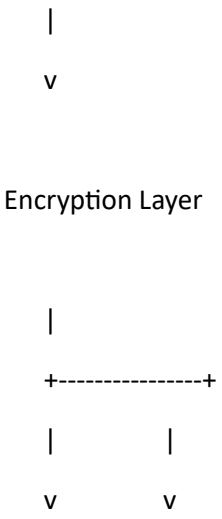
- AI API keys
- local AI connection settings
- private configuration
- external service credentials

while keeping deployment simple.

---

**Security Architecture**

User Settings




---

**Protected data**

Examples:

AI Provider

OpenAI

├── API Key 🔒

├── Model

└── Parameters

Claude

├── API Key 🔒

Gemini

├── API Key 🔒

KoboldCpp

├── Endpoint

└── Local settings

---

**Security features**

- encrypted database fields
- no plaintext API keys
- environment variable support
- safe Docker configuration
- export sanitization

Example:

When exporting a workspace:

Before:

api_key:

sk-xxxxxxxx

After:

api_key:

[REMOVED]

---

**Question 29 — ComfyUI Communication Method**

For ComfyUI integration, how should the connection work?

**A. Simple REST/WebSocket connection** ⭐

Connect to a running ComfyUI instance.

Example:

Wildcard Studio

|

|

v

ComfyUI API

localhost:8188

Features:

- send workflows
- receive images
- monitor queue
- retrieve history

---

**B. Install ComfyUI inside the same Docker stack**

The application includes:

docker-compose.yml

├── Wildcard Studio

├── PostgreSQL

└── ComfyUI

Advantages:

- one-click environment

Disadvantages:

- huge image
- GPU configuration complexity

---

**C. Support both**

Default:

- external ComfyUI connection

Optional:

- bundled ComfyUI container profile

---

Which option do you want?

A, B, or C?

A

Excellent. This keeps the application lightweight and respects the separation of responsibilities.

**ComfyUI communication decision confirmed**

The project will use a **connector-based integration with an external ComfyUI instance**.

Wildcard Management Studio will act as the **prompt intelligence layer**, while ComfyUI remains the **generation engine**.

---

**ComfyUI Integration Architecture**

Wildcard Management Studio

|

|

v

ComfyUI Connector Module

|

|

v

Running ComfyUI Instance

(local network / localhost)

|

|

v

Generated Images + Metadata

---

**Supported communication**

The connector will support:

- REST API communication
- WebSocket monitoring
- workflow submission
- queue monitoring
- generation status
- image retrieval
- metadata extraction

---

**Features**

**Send generation requests**

The IDE can send:

Positive Prompt

Negative Prompt

Seed

Steps

CFG

Sampler

Scheduler

Checkpoint

LoRAs

Workflow JSON

---

**Workflow association**

A prompt can remember:

Prompt:

Cyberpunk Warrior

Associated Workflow:

Illustrious Character Workflow.json

Model:

Illustrious XL

Preferred Sampler:

DPM++ 2M

---

**Generation tracking**

After generation:

Generated Image

linked to:

Prompt Version

Wildcard Expansion

Seed

Workflow

Model

Date

---

**Question 30 — Diffusion Model Support Scope**

You already specified:

- SDXL
- Illustrious
- NoobAI
- Anima
- Ideogram 4

Should the first version also include profiles for other popular models?

**A. Only the listed models**

Initial support:

- SDXL
- Illustrious
- NoobAI
- Anima
- Ideogram 4

Keep scope controlled.

---

**B. Add major current models** ⭐

Include:

- FLUX
- Stable Diffusion 3.x
- Pony Diffusion
- Hunyuan Image
- PixArt
- other popular models

---

**C. Generic model framework only**

No assumptions.

Users create all model profiles themselves.

---

**D. Hybrid** ⭐ ⭐

Ship with your listed models + several popular models, while keeping the editable profile system.

---

Which one do you choose?

A, B, C, or D?

D

Excellent. This is the best choice for a long-lived Prompt IDE.

**Diffusion model support decision confirmed**

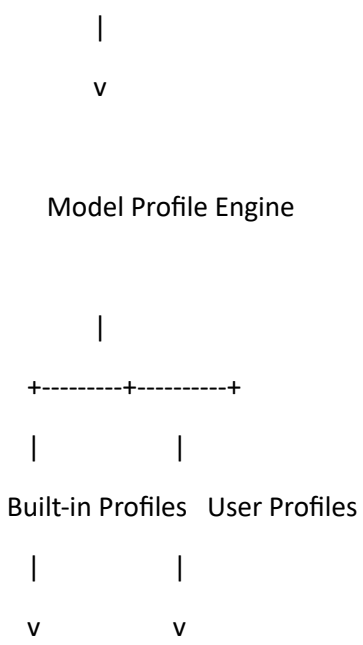
The project will use a **hybrid model profile system**:

- ✅ Built-in profiles for your priority models
- ✅ Additional popular model profiles included
- ✅ Fully editable by the user
- ✅ New models can be added through the extension system

---

**Model Support Architecture**

Model Knowledge System



SDXL Custom Model

Illustrious Experimental Model

NoobAI

Anima

Ideogram 4

FLUX

SD3.x

Pony

Hunyuan

PixArt

...

---

**Built-in profiles**

Initial release:

**Required profiles**

- SDXL
- Illustrious
- NoobAI
- Anima
- Ideogram 4

**Additional profiles**

Potentially include:

- FLUX family
- Stable Diffusion 3.x
- Pony Diffusion
- Hunyuan Image
- PixArt
- future models

---

**Each model profile contains**

Model Profile

Name

Version

Prompt style

Tag philosophy

Natural language preference

Recommended prompt order

Weighting syntax

Wildcard behavior

Negative prompt rules

Quality modifiers

Examples

AI conversion instructions

RAG references

---

**AI conversion example**

Source:

SDXL

"A warrior woman in a dark forest"

Target:

Illustrious

1girl, warrior, armor, sword,

dark forest, detailed eyes

The conversion uses the selected model profile.

---

**Question 31 — Prompt Evaluation and Scoring**

Would you like the application to evaluate prompts automatically?

Example:

Prompt Quality Analysis

Model compatibility: 92%

Structure: 85%

Specificity: 90%

Wildcard diversity: 75%

Redundancy: 15%

Potential improvements:

- Add lighting information

- Reduce repeated quality tags

Options:

**A. No scoring system**

Only editing and organization.

---

**B. Basic prompt analyzer**

Checks:

- length
- duplicates
- missing sections
- syntax errors

---

**C. AI-powered prompt evaluation** ⭐

Uses:

- model profile
- RAG knowledge
- tag ontology
- AI reasoning

to provide meaningful feedback.

---

**D. Full benchmark laboratory**

Everything in C plus:

- compare prompt versions
- compare models
- track generation results
- learn from user ratings
- optimize prompts automatically over time

---

Which option do you want?

A, B, C, or D?

C

Excellent. This keeps the scope powerful while staying focused on prompt engineering rather than becoming a full automatic optimization research platform.

**Prompt evaluation decision confirmed**

The project will include an **AI-powered Prompt Evaluation System**.

The objective:

Help users understand why a prompt works, where it can be improved, and how it fits the selected generation model.

---

**Prompt Evaluation Architecture**

Prompt

|

|

v

Analysis Pipeline

|

+----------------+

|  |

v  v

Tag Analysis  Model Profile

|  |

+----------------+

|

v

RAG Knowledge

|

v

AI Evaluation

|

v

Prompt Report

---

**Evaluation categories**

**Model compatibility**

Example:

Target:

Illustrious XL

Score:

92%

Analysis:

✓ Good character tags

✓ Correct tag ordering

⚠ Missing quality modifiers

---

**Structure analysis**

Checks:

- subject clarity
- character definition
- environment
- action
- composition
- style
- lighting

---

**Tag analysis**

Checks:

- duplicate tags
- conflicting tags
- outdated tags
- incorrect model usage

Example:

Warning:

"photorealistic" may conflict with anime-oriented model profile

---

**Wildcard analysis**

Checks:

- missing dependencies
- low diversity
- excessive repetition
- unbalanced distributions

---

**Improvement suggestions**

Example:

Original:

girl in forest

AI suggests:

Add:

- character details

- lighting

- composition

- atmosphere

---

**Question 32 — AI Provider Priority**

You want support for:

- KoboldCpp
- ChatGPT
- Claude
- Gemini

Now we need to define how the application chooses which AI to use.

**A. User manually selects provider every time**

Example:

"Use Claude"

"Use KoboldCpp"

---

**B. One default AI provider**

User chooses:

Default AI:

KoboldCpp

Everything uses it unless changed.

---

**C. Task-based AI routing** ⭐

Different tasks use different models.

Example:

Prompt rewriting:

Claude

Tag conversion:

KoboldCpp

RAG search:

Local model

Translation:

Gemini

---

**D. Intelligent automatic routing**

The system decides:

- local model if possible
- cloud model when needed
- cheapest/fastest option

---

Which option do you want?

A, B, C, or D?

B with possibility of C

Excellent. This is a very practical approach.

**AI provider strategy decision confirmed**

The application will use:

✅ **One default AI provider** as the primary mode
✅ **Task-based routing capability** as an advanced option

This gives simplicity for normal use while allowing power users to fine-tune workflows.

---

**AI Provider Architecture**

AI Configuration

|

v

Default Provider

|

+----------+

| |

v v

Simple Mode Advanced Routing Mode

All tasks Task-specific models

|

+----------+----------+

| | |

v v v

Prompt Edit Translation Analysis

Claude Gemini KoboldCpp

---

**Default AI Mode**

Example:

Default AI Provider:

KoboldCpp Local

Model:

Llama-based model

Temperature:

0.7

All AI features use this provider.

---

**Advanced Task Routing**

Optional configuration:

ai_routing:

prompt_improvement:

provider: claude

wildcard_generation:

provider: koboldcpp

translation:

```
  provider: gemini


 documentation_analysis:
  provider: local_rag
```

---

**Supported AI connectors**

Initial connectors:

- KoboldCpp
- OpenAI-compatible APIs
- OpenAI
- Anthropic Claude
- Google Gemini

Future through extension system:

- Ollama
- LM Studio
- vLLM
- other providers

---

**Question 33 — Prompt Collaboration / Sharing**

Even though this is a single-user application, should prompts be shareable?

**A. No sharing features**

Only local usage.

---

**B. Export/import sharing** ⭐

Users can share:

- prompts
- wildcards
- workspaces
- model profiles

Formats:

- .prompt

- YAML
- ZIP workspace package

---

**C. Online sharing platform**

A future community system:

- upload prompts
- browse others
- ratings
- comments
- collections

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the application local-first while still allowing users to exchange useful assets.

**Sharing decision confirmed**

The project will support **local export/import sharing**.

The philosophy:

Users own their prompt ecosystem and can move, archive, or share selected assets without requiring an online platform.

---

**Sharing & Export Architecture**

Workspace Assets

Prompts

Wildcards

Tags

Model Profiles

Workflows

Knowledge Documents

Images Metadata

|

v

Export Package

|

v

Import

|

v

Merge / Replace / Duplicate Detection

---

**Exportable assets**

**Prompt packages**

Example:

Cyberpunk_Warrior.promptpack

Contains:

prompt.prompt

metadata.yaml

wildcard references

model profile

preview image

notes

---

**Wildcard packs**

Example:

Fantasy_Wildcards.zip

Contains:

characters.yaml

clothing.txt

locations/

Compatible with:

- ComfyUI wildcard workflows
- Wildcard Management Studio

---

**Workspace export**

Example:

My_Project.wsp

Contains:

prompts/

wildcards/

tags/

models/

settings/

metadata/

---

**Import intelligence**

When importing:

The system checks:

- duplicate prompts
- duplicate wildcards
- conflicting tags
- missing dependencies
- incompatible models

Example:

Import result:

✓ 120 prompts added

✓ 45 wildcards merged

⚠ 12 duplicates detected

⚠ 3 missing references

---

**Question 34 — Knowledge Base Management**

For the RAG system containing:

- SDXL guides
- Illustrious guides
- Anima guides
- Ideogram guides
- user documentation

How should users manage knowledge documents?

**A. Built-in knowledge only**

Users cannot add documents.

---

**B. User-managed knowledge base** ⭐

Users can import:

- PDF
- Markdown
- TXT
- web articles
- notes

The system indexes them.

---

**C. Advanced knowledge workspace**

Everything in B plus:

- document collections
- source tracking
- versioning
- per-model knowledge spaces
- custom AI assistants

---

Which one do you want?

A, B, or C?

B

Excellent. This matches the overall philosophy: **local ownership, extensibility, and user-controlled intelligence**.

**Knowledge base decision confirmed**

The application will include a **user-managed local RAG knowledge base**.

The user controls what knowledge the AI uses.

---

**Knowledge Base Architecture**

Knowledge Workspace

Documents

|

v

Parser Engine

|

v

Text Chunking

|

v

Embedding Generator

|

v

PostgreSQL + pgvector

|

v

AI Assistant Retrieval

---

**Supported imports**

The knowledge system can ingest:

- Markdown (.md)
- Text (.txt)
- PDF documents
- YAML documentation
- HTML/web exports
- personal notes
- prompt examples

---

**Knowledge organization**

Users can create collections:

Example:

Knowledge Base

├── SDXL
│ ├── prompting guide.md
│ └── examples.md
│
├── Illustrious
│ ├── tag rules.md
│ └── character prompting.md
│
├── Anima

│ └── references.md

│

└── Personal Research

└── my_notes.md

---

**AI retrieval behavior**

When asking:

"Improve this prompt for Illustrious"

The AI retrieves:

Selected Model:

Illustrious

Relevant Knowledge:

✓ Illustrious tag ordering guide

✓ Character prompt examples

✓ User preferences

✓ Previous successful prompts

---

**Document metadata**

Each document stores:

Document

Title

Source

Author

Date

Model association

Tags

Version

Embedding status

---

**Question 35 — Prompt Creation Workflow**

How should users create new prompts?

**A. Text-first**

User starts with:

empty editor

and writes manually.

AI helps afterward.

---

**B. AI-first assistant**

User describes an idea:

"Create a fantasy mage portrait"

AI generates the prompt structure.

---

**C. Hybrid** ⭐

Both workflows:

- write manually
- ask AI
- use templates
- combine existing assets
- generate from images
- convert from other models

---

Which one do you want?

A, B, or C?

C

Excellent. This is the natural choice for a Prompt IDE.

**Prompt creation workflow decision confirmed**

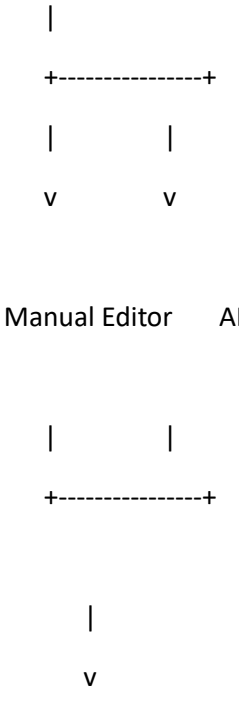
The system will support a **hybrid prompt creation workflow**.

The application should feel like a combination of:

- VS Code (manual creation)
- Photoshop/Blender (creative building blocks)
- Copilot-style AI assistance
- prompt engineering laboratory

---

**Prompt Creation Architecture**



|

v

Model Optimization

|

v

Saved Prompt

---

**Creation methods**

**1. Manual editing**

Using Monaco:

masterpiece, __character__, cinematic lighting

With:

- autocomplete
- syntax highlighting
- wildcard suggestions
- tag suggestions

---

**2. AI generation**

User:

"Create a dark fantasy warrior portrait"

AI creates:

subject:

female warrior

appearance:

black armor

silver hair

environment:

ruined castle

style:

dark fantasy

cinematic lighting

---

**3. Template system**

Reusable templates:

Portrait Template

Character:

—

Appearance:

—

Environment:

—

Lighting:

—

Style:

—

---

**4. Image-based creation**

Using DanTagGen:

Image

↓

Tags

↓

Prompt

↓

Optimization

---

**5. Model conversion**

Example:

SDXL Prompt

↓

Illustrious Prompt

↓

Anima Prompt

---

**Question 36 — ComfyUI Wildcard Simulation Accuracy**

One important technical point:

The application needs to simulate wildcard expansion. How close should the simulation be to actual ComfyUI behavior?

**A. Approximate simulation**

The application expands wildcards for preview purposes.

Fast but may not perfectly match ComfyUI.

---

**B. Exact Impact Pack compatibility** ⭐

Reproduce ComfyUI Impact Wildcard behavior:

- same syntax
- same nesting rules
- same random selection logic

- same YAML/TXT interpretation

The simulator becomes a test environment.

---

**C. Dual mode**

Provide:

- exact compatibility mode
- enhanced IDE simulation mode

Example:

ComfyUI Compatible Mode

+

Creative Testing Mode

---

Which one do you want?

A, B, or C?

B

Excellent. This is a critical decision because wildcard behavior is the foundation of the project.

**Wildcard simulation decision confirmed**

The application will implement **exact ComfyUI Impact Pack wildcard compatibility**.

The goal:

A prompt tested inside Wildcard Management Studio should behave the same way when executed inside ComfyUI.

---

**Wildcard Engine Architecture**

ComfyUI Wildcard Engine

|

v

Syntax Parser

|

v

Expansion Resolver

|

+----------------+

| |

v v

TXT Files YAML Files

|

v

Recursive Expansion

|

v

Final Prompt Output

---

**Compatibility requirements**

The engine will support:

- __wildcard_name__
- nested wildcards
- recursive expansion

- wildcard file discovery
- TXT format
- YAML format
- random selection
- hierarchy resolution
- ComfyUI directory conventions

---

**Simulation example**

Input:

portrait of __character__ wearing __clothing__ in __location__

Wildcard:

character:

- elf warrior
- cyber knight

clothing:

- enchanted armor
- leather jacket

location:

- ancient forest
- futuristic city

Simulation:

Run #1

elf warrior

wearing enchanted armor

in ancient forest

Run #2

cyber knight

wearing leather jacket

in futuristic city

---

**Testing features**

The simulator will support:

- deterministic seeds
- random seeds
- batch expansion
- expansion preview
- dependency tree
- missing wildcard detection
- recursive loop detection

---

**Important implementation note**

The final development prompt should instruct the developer/AI agent to:

- study the Impact Pack wildcard specification
- implement compatibility tests
- create sample wildcard packs
- verify output against ComfyUI behavior

---

**Question 37 — Prompt Data Import Sources**

Besides TXT/YAML wildcard files, should the system import prompts from external sources?

Possible sources:

- CivitAI metadata
- PNG metadata from generated images
- A1111/Forge metadata
- ComfyUI workflows
- JSON exports
- Markdown collections

**A. Only TXT/YAML/manual input**

Keep scope limited.

---

**B. Common AI image sources** ⭐

Support:

- PNG metadata
- ComfyUI workflows
- Automatic1111/Forge parameters
- JSON prompt exports

---

**C. Full ecosystem importer**

Everything in B plus:

- CivitAI imports
- image galleries
- prompt databases
- external collections

---

Which one do you want?

A, B, or C?

C

Excellent. This reinforces the vision of the application as a **central knowledge hub for AI image prompting**.

**Prompt data import decision confirmed**

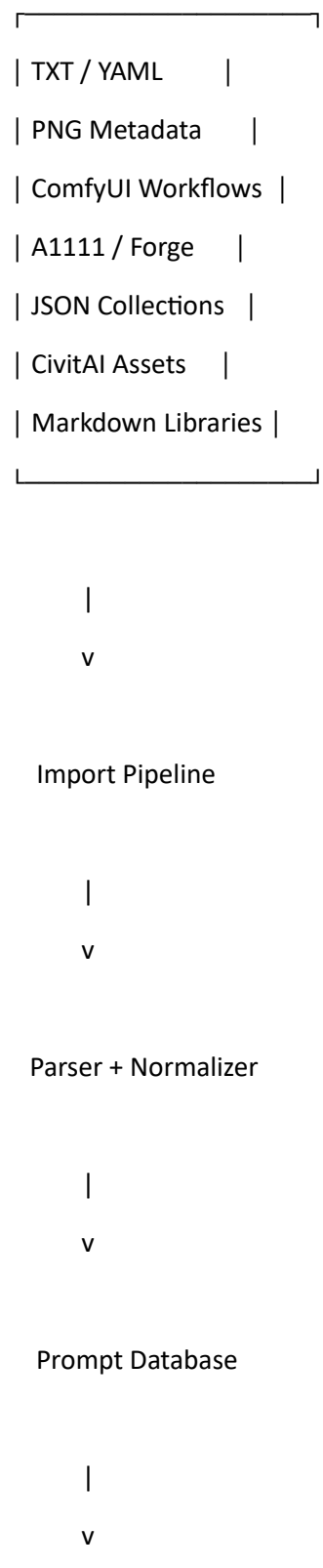
The system will support a **full ecosystem importer**.

The objective:

Bring existing prompt assets, generated images, workflows, and knowledge collections into one structured Prompt IDE database.

---

**Import Ecosystem Architecture**



AI Analysis / Tagging / RAG

---

**Supported import sources**

**1. Wildcards**

Formats:

- TXT
- YAML

Features:

- parse
- normalize
- index
- preserve source files

---

**2. Generated images**

Extract from:

- PNG metadata
- ComfyUI metadata
- A1111 metadata
- Forge metadata

Retrieve:

Prompt

Negative Prompt

Seed

Model

Sampler

Steps

CFG

LoRAs

Workflow

---

**3. ComfyUI workflows**

Import:

- workflow JSON
- node information
- model references
- prompt nodes

Associate:

Prompt

+

Workflow

+

Generated Images

---

**4. CivitAI ecosystem**

Import:

- prompts
- model information
- examples
- metadata
- references

(with user-controlled import, respecting source information)

---

**5. Existing prompt libraries**

Support:

- Markdown collections
- JSON exports
- custom folders
- CSV datasets

---

**Import intelligence**

Before adding data:

The system analyzes:

Import Preview

Found:

2,340 prompts

450 wildcards

18,000 tags

920 images

Duplicates:

340

Missing metadata:

120

User chooses:

- import all

- merge
- skip duplicates
- review conflicts

---

**Question 38 — Copyright and Licensing Management**

You requested organization by copyright.

How advanced should copyright tracking be?

**A. Simple field**

Each asset has:

Copyright:

Original

Unknown

Other

---

**B. Structured licensing system** ⭐

Support:

License

CC0

CC-BY

Public Domain

Commercial Allowed

Commercial Restricted

Personal Use Only

Unknown

With:

- author
- source URL
- notes

---

**C. AI-assisted copyright analysis**

Everything in B plus:

AI attempts to detect:

- artist names
- copyrighted characters
- style references
- potential restrictions

Example:

Detected:

Possible character:

Batman

Possible artist reference:

Unknown

Review recommended

---

Which one do you want?

A, B, or C?

C

Excellent. This is an important feature for a serious prompt management system, especially as prompt libraries grow and include imported assets.

**Copyright management decision confirmed**

The project will implement an **AI-assisted copyright and licensing management system**.

The goal:

Help users understand the origin, usage conditions, and potential risks of prompts, images, styles, and imported assets.

---

**Copyright & Licensing Architecture**

Asset

(Prompt / Image / Wildcard / Model Profile)

|

v

Metadata Analysis

|

+----------+

| |

v v

Manual Data AI Analysis

|

v

Licensing Database

|

v

Copyright Report

---

**License management**

Each asset can contain:

Copyright Metadata

Author

Source

License

Creation Date

Reference URL

Commercial Usage

Notes

Confidence Level

---

**Supported licenses**

Examples:

CC0

CC-BY

Public Domain

Commercial Allowed

Commercial Restricted

Personal Use Only

Unknown

---

**AI analysis capabilities**

The system can analyze:

**Prompt content**

Detect:

- named characters
- franchises
- brands
- known entities

Example:

Prompt:

"Batman standing in Gotham"

Detection:

Character:

Batman

Category:

Copyrighted character

Review:

Recommended

---

**Style references**

Detect:

- artist names
- recognizable styles
- model-specific restrictions

Example:

Detected:

Possible artist reference:

Unknown

Style reference:

Anime illustration

Action:

Review metadata

---

**Imported assets**

When importing:

The system creates:

Asset Report

Source:

CivitAI

Author:

Detected

License:

CC-BY

Risk:

Low

---

**User control**

The AI does not block content.

It provides:

- warnings
- metadata suggestions
- organization help
- traceability

---

**Question 39 — Prompt Generation Output Targets**

When the AI creates or converts prompts, which output formats should it prioritize?

**A. Only prompt text**

Example:

masterpiece, detailed eyes, forest...

---

**B. Structured output** ⭐

Generate:

- .prompt
- YAML
- sections
- metadata
- wildcards
- model profile

Example:

subject:

style:

environment:

wildcards:

model:

---

**C. Multi-target generation**

The AI can create:

Same concept:

SDXL version

Illustrious version

Anima version

Ideogram version

ComfyUI version

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the system focused on being a **professional prompt engineering environment** rather than just a text generator.

**Prompt output decision confirmed**

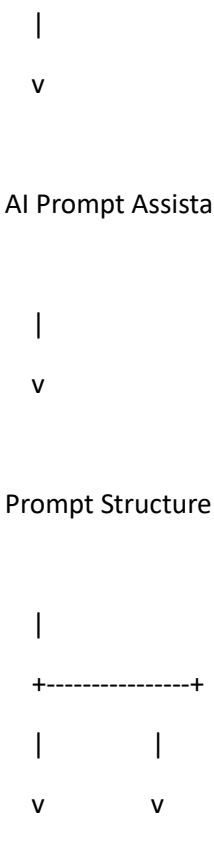
The application will prioritize **structured prompt generation**.

The .prompt format becomes the canonical editable format, with conversion/export to other formats when needed.

---

**Structured Prompt Generation Architecture**

User Idea




---

**Generated prompt structure**

Example:

```
name: Dark Fantasy Warrior

metadata:
  theme:
    - fantasy
    - dark
  copyright:
```

```
  status: unknown

model:
 target:
  - Illustrious

prompt:

 subject:
  - female warrior
  - knight

 appearance:
  - black armor
  - silver hair

 action:
  - standing

 environment:
  - ruined castle

 lighting:
  - cinematic lighting

 style:
  - dark fantasy

wildcards:

 - __character__
```

- __weapon__

negative:

- low quality
- blurry

---

**Benefits**

The AI can now:

- understand each prompt component
- reorder sections per model
- convert models
- analyze weaknesses
- compare versions
- create wildcards
- generate documentation

---

**Prompt editing experience**

Monaco editor features:

- YAML-like syntax highlighting
- autocomplete
- tag suggestions
- wildcard validation
- model-specific hints
- inline AI suggestions

---

**Question 40 — Database Technology**

The application requires:

- prompts
- tags
- embeddings

- images metadata
- versions
- knowledge base
- AI memory

Which database architecture should be used?

**A. PostgreSQL + pgvector** ⭐

Relational database + vector search.

Supports:

- structured data
- semantic search
- RAG
- scalability

---

**B. SQLite + local vector extension**

Simpler:

- single file database
- easy backup
- lightweight

---

**C. Hybrid**

Use:

- PostgreSQL for main database
- local cache/indexes
- filesystem assets

---

Which one do you want?

A

Excellent. This is the correct choice for the scale and ambition of the project.

**Database architecture decision confirmed**

The project will use:

**PostgreSQL + pgvector**

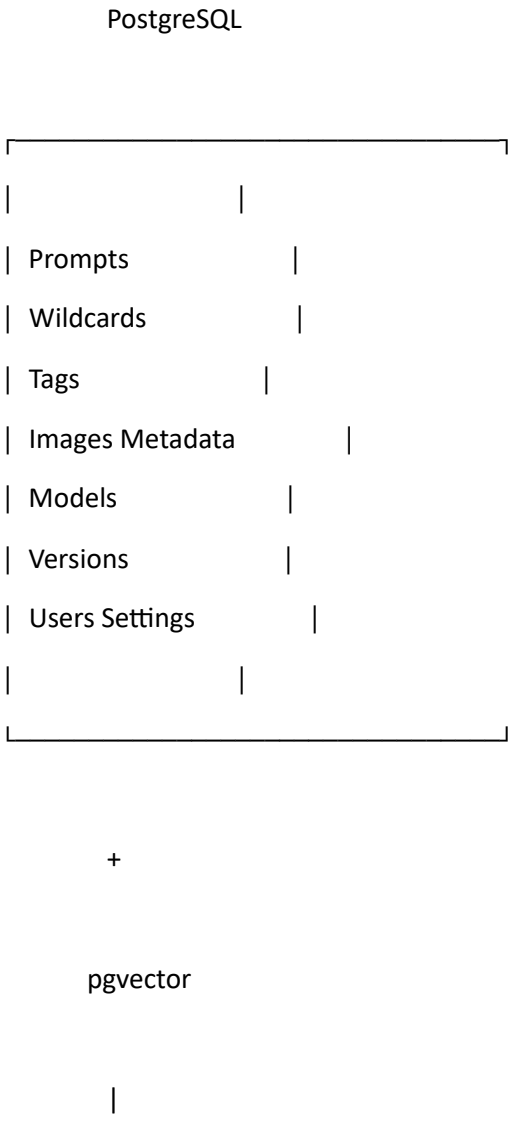
as the primary database engine.

This provides:

- ✅ structured relational storage
- ✅ semantic search
- ✅ RAG knowledge retrieval
- ✅ scalable prompt management
- ✅ complex relationships between prompts, tags, images, models, and versions

---

**Database Architecture**



Semantic Embeddings

|

AI Search / RAG

---

**Core database entities**

**Prompts**

Stores:

- .prompt content
- YAML representation
- metadata
- model compatibility
- versions
- ratings
- embeddings

---

**Wildcards**

Stores:

- source file
- parsed entries
- hierarchy
- dependencies
- expansion statistics

---

**Tags**

Stores:

- tag name
- aliases
- ontology relationships
- model compatibility

- embeddings

---

**Images**

Stores metadata:

- filename
- location
- prompt
- seed
- model
- workflow
- tags
- similarity embedding

---

**Model Profiles**

Stores:

- prompting rules
- examples
- AI instructions
- knowledge references

---

**Knowledge Base**

Stores:

- documents
- chunks
- embeddings
- sources
- model association

---

**Semantic search examples**

User searches:

"female warrior in dark fantasy"

Database finds:

Prompt:

shadow knight

Tags:

female warrior

dark fantasy

armor

Image:

similar composition

---

**Question 41 — Docker Architecture**

You requested that the complete project runs in Docker.

How should the Docker deployment be structured?

**A. Simple single container**

Everything inside:

Wildcard Studio Container

Pros:

- easiest deployment

Cons:

- poor separation
- harder maintenance

---

**B. Docker Compose multi-service** ⭐

Example:

services:

frontend:

React/Vue

backend:

API

database:

PostgreSQL + pgvector

worker:

AI/indexing jobs

Pros:

- professional architecture
- scalable
- easier development

---

**C. Kubernetes-ready architecture**

Everything in B plus:

- cloud deployment
- scaling
- orchestration

---

Which one do you want?

A, B, or C?

B

Excellent. This is the right architecture for the project.

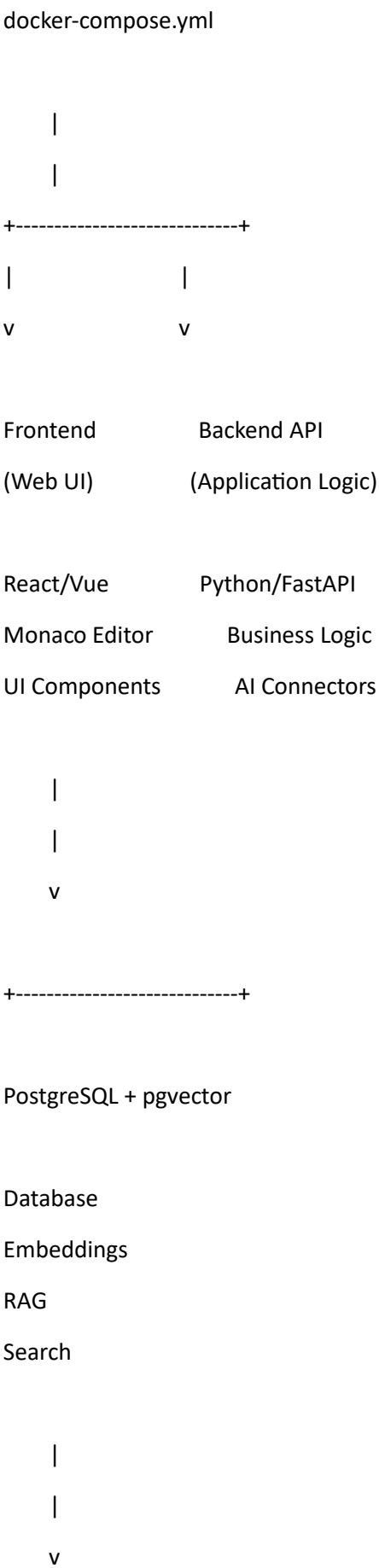
**Docker architecture decision confirmed**

The project will use a **Docker Compose multi-service architecture**.

This provides a professional development environment while remaining easy to install locally.

---

**Docker Architecture**


docker-compose.yml
+----------+
| |
v v
Frontend Backend API
(Web UI) (Application Logic)
React/Vue Python/FastAPI
Monaco Editor Business Logic
UI Components AI Connectors
+----------+
PostgreSQL + pgvector
Database
Embeddings
RAG
Search

Background Worker

Import Processing

AI Tasks

Indexing

Metadata Extraction

---

**Service responsibilities**

**Frontend container**

Handles:

- graphical web interface
- Monaco editor
- prompt editor
- explorer panel
- settings UI
- search interface
- visualization tools

---

**Backend container**

Handles:

- API
- authentication/local security
- prompt processing
- wildcard engine
- ComfyUI connector
- AI provider connectors
- model profiles
- version management

---

**Database container**

PostgreSQL + pgvector:

Handles:

- relational data
- embeddings
- semantic search
- RAG retrieval

---

**Worker container**

Handles heavy tasks:

- file imports
- DanTagGen processing
- document indexing
- embedding generation
- batch analysis
- wildcard simulations

---

**Persistent volumes**

Example:

volumes:

postgres_data/

workspace_data/

backup_data/

model_profiles/

knowledge_base/

---

**Development benefits**

The system can later support:

- local installation
- NAS deployment
- workstation deployment
- server deployment

without changing the architecture.

---

**Question 42 — Frontend Technology**

The graphical web interface is a major part of the project.

Which frontend approach should be used?

**A. React + TypeScript** ⭐

Industry standard.

Advantages:

- huge ecosystem
- Monaco Editor integration
- many UI libraries
- good long-term support

---

**B. Vue + TypeScript**

Advantages:

- simpler learning curve
- excellent UI development
- clean architecture

---

**C. Svelte / SvelteKit**

Advantages:

- lightweight
- modern
- very fast

---

**D. Angular**

Advantages:

- enterprise architecture
- strict structure

---

Which one do you choose?

A, B, C, or D?

A

Excellent. This aligns very well with the IDE-style interface requirement.

**Frontend technology decision confirmed**

The project will use:

**React + TypeScript**

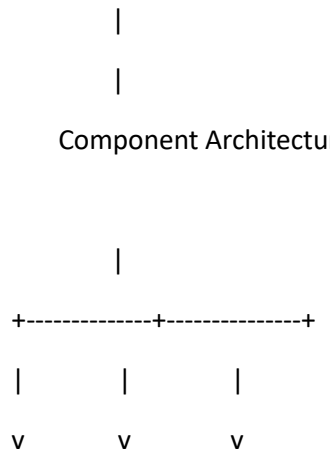
for the graphical web user interface.

The frontend will be designed as a professional creative IDE, with architecture inspired by tools like VS Code.

---

**Frontend Architecture**

React + TypeScript Application



Monaco Files Settings

Prompt IDE Workspace Search

Assets Analysis

---

**Main UI Components**

**Main Workspace**

VS Code-style layout:

```
+-------------------------------------------------+
| Menu / Toolbar                                  |
+---------------------+---------------------------+
|                     |                           |
| Explorer            | Monaco Prompt Editor      |
|                     |                           |
| - Prompts           |                           |
| - Wildcards         |                           |
| - Images            |                           |
| - Models            |                           |
|                     |                           |
+---------------------+---------------------------+
| AI Assistant / Analysis / Preview               |
+-------------------------------------------------+
```

---

**Monaco Editor integration**

Features:

- .prompt syntax highlighting
- YAML editing
- wildcard autocomplete
- tag suggestions
- AI inline assistance
- diff viewer

- version comparison

---

**UI component system**

Will support:

- reusable components
- dark/light themes
- responsive panels
- customizable layouts

---

**Recommended frontend stack**

The final development prompt should specify:

- React
- TypeScript
- Monaco Editor
- modern component framework
- state management
- API communication layer
- WebSocket support for live updates

---

**Question 43 — Backend Technology**

The backend will handle:

- database communication
- wildcard engine
- AI providers
- ComfyUI connector
- import system
- RAG
- background jobs

Which backend framework should be used?

**A. Python + FastAPI** ⭐

Advantages:

- excellent AI ecosystem
- native ML tooling
- async support
- easy API creation
- ideal for ComfyUI integration

---

**B. Node.js + TypeScript**

Advantages:

- same language as frontend
- large ecosystem

---

**C. Python Django**

Advantages:

- mature framework
- built-in admin
- batteries included

---

**D. Go backend**

Advantages:

- fast
- efficient
- strong concurrency

---

Which one do you choose?

A, B, C, or D?

A

Excellent. This is the most appropriate choice for this project because a large part of the system relies on AI, parsing, embeddings, image analysis, and ComfyUI integration.

**Backend technology decision confirmed**

The project will use:

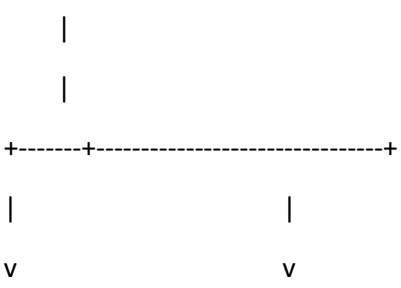
**Python + FastAPI**

as the backend framework.

---

**Backend Architecture**

FastAPI Backend




---

**Backend modules**

**Prompt Engine**

Handles:

- .prompt parsing
- YAML conversion
- prompt generation
- model conversion
- validation

---

**Wildcard Engine**

Handles:

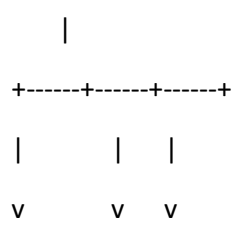
- ComfyUI Impact Pack compatibility
- TXT/YAML parsing
- recursive expansion
- simulation
- statistics

---

**AI Service Layer**

Unified interface:

AI Provider Interface




Allows switching providers without changing application logic.

---

**RAG Engine**

Handles:

- document ingestion
- chunking
- embeddings
- pgvector queries
- AI knowledge retrieval

---

**Import Pipeline**

Handles:

- TXT
- YAML
- PNG metadata
- ComfyUI workflow JSON
- A1111/Forge metadata
- CivitAI data

---

**Background Processing**

Recommended:

- Celery or similar task queue
- Redis message broker

For:

- large imports
- DanTagGen jobs
- embedding generation
- batch analysis

---

**Question 44 — Background Processing System**

The project will need background tasks for:

- importing thousands of files
- DanTagGen image analysis
- embedding generation
- wildcard simulation batches
- document indexing

How should this work?

**A. Simple FastAPI background tasks**

Built into FastAPI.

Good for:

- small operations

---

**B. Professional worker queue** ⭐

Use:

- Redis
- Celery/RQ
- dedicated worker container

Architecture:

API

|

v

Redis Queue

|

v

Worker Container

|

+-- Imports

+-- AI Tasks

+-- Indexing

---

**C. Distributed processing system**

Everything in B plus:

- multiple workers
- GPU workers
- workload scheduling

---

Which one do you want?

A, B, or C?

B

Excellent. This is the correct choice for a serious local AI application.

**Background processing decision confirmed**

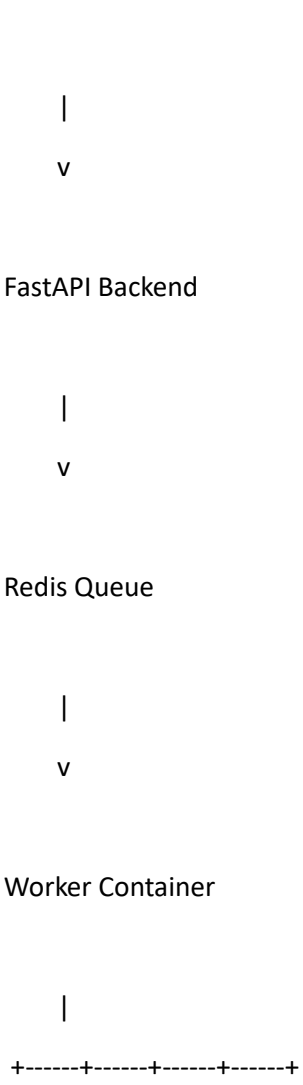
The project will use a **professional worker queue architecture**.

The goal:

Keep the user interface responsive while heavy operations run in the background.

---

**Background Processing Architecture**




---

**Worker tasks**

**Import processing**

Handles:

- TXT parsing
- YAML parsing
- PNG metadata extraction
- workflow imports
- CivitAI imports
- duplicate detection

---

**AI image analysis**

Handles:

- DanTagGen execution
- tag extraction
- tag normalization
- ontology matching

---

**Knowledge indexing**

Handles:

- PDF parsing
- document chunking
- embeddings
- pgvector insertion

---

**Wildcard simulation**

Handles:

- batch expansion
- statistics
- diversity analysis
- optimization reports

---

**User experience**

Example:

User imports:

50,000 images

Instead of freezing:

Import started

 60%

Processing:

12,500 / 50,000 images

Current task:

DanTagGen analysis

---

**Question 45 — Authentication / User Management**

Because this is primarily a local application, how should users be managed?

**A. No authentication**

Local single-user application.

Open directly after startup.

---

**B. Local user accounts** ⭐

Support:

- username/password
- user preferences
- separate workspaces
- multiple users on same installation

---

**C. Full authentication system**

Everything in B plus:

- OAuth
- LDAP
- remote access security

- enterprise features

---

Which one do you want?

A, B, or C?

A

Excellent. This keeps the application aligned with its intended use: a **local-first creative development environment**.

**Authentication decision confirmed**

The project will use **no user authentication system**.

The application will assume a trusted local environment.

---

**Local-Only Security Model**

User Browser

|

v

Wildcard Management Studio

|

+----------+

| |

v v

Local Files Local Database

---

**Design implications**

**No login screen**

The user opens:

http://localhost:xxxx

and immediately accesses the workspace.

---

**Security focus shifts to:**

- encrypted API keys
- local database protection
- Docker isolation
- safe exports
- workspace permissions

---

**Multi-user separation**

Not included in the first version.

Future extension possibility:

- optional accounts
- remote server mode
- team collaboration

---

**Question 46 — Workspace and File Synchronization**

Since the application uses real workspace folders, how should file changes be handled?

Example:

A user edits:

wildcards/characters.yaml

outside the application.

Should the IDE detect and update automatically?

---

**A. Manual refresh**

User presses:

"Reload workspace"

Simple.

---

**B. Automatic file watching** ⭐

The application monitors workspace folders.

Detects:

- new files
- changes
- deletions
- renames

and updates the database automatically.

---

**C. Full synchronization engine**

Everything in B plus:

- conflict resolution
- offline change tracking
- merge tools
- external editor awareness

---

Which one do you want?

A, B, or C?

B

Excellent. This fits perfectly with the VS Code-style workspace approach you selected earlier.

**Workspace synchronization decision confirmed**

The project will implement **automatic file watching**.

The database becomes a live index of the workspace while the filesystem remains the source of truth.

---

**Workspace File Synchronization Architecture**

Workspace Folder

(prompts/

wildcards/

images/

workflows/)

|

v

File Watcher Service

|

+----------------+

| |

v v

Change Detector Parser Pipeline

|

v

PostgreSQL Index Update

---

**Detected events**

The system monitors:

**New files**

Example:

New file detected:

wildcards/weapons.yaml

Action:

- parse file
- validate syntax

- import entries
- update database

---

**Modified files**

Example:

characters.yaml changed

Action:

- compare versions
- update index
- create version snapshot

---

**Deleted files**

Example:

old_prompt.prompt removed

Action:

- mark asset unavailable
- preserve history
- offer restore

---

**Renamed files**

Preserve:

- relationships
- versions
- metadata

---

**Supported external editors**

Because files remain standard:

Users can edit with:

- Wildcard Management Studio
- VS Code
- any text editor

Changes synchronize automatically.

---

**Question 47 — Search Index Update Strategy**

With PostgreSQL + pgvector and automatic file watching, when should the search index update?

**A. Immediately after every change**

Real-time indexing.

Pros:

- always current

Cons:

- heavier for huge libraries

---

**B. Background indexing** ⭐

Changes go into a queue.

Example:

File changed

↓

Queue

↓

Worker indexes later

Pros:

- smooth UI
- handles large projects

---

**C. Manual indexing**

User controls when indexing happens.

---

Which one do you want?

A, B, or C?

B

Excellent. This is the right balance for a large prompt library.

**Search indexing decision confirmed**

The project will use **background indexing through the worker system**.

The interface remains responsive while the system updates search, embeddings, and AI knowledge in the background.

---

**Search Indexing Architecture**

File Change Detected

|

v

File Watcher

|

v

Index Queue (Redis)

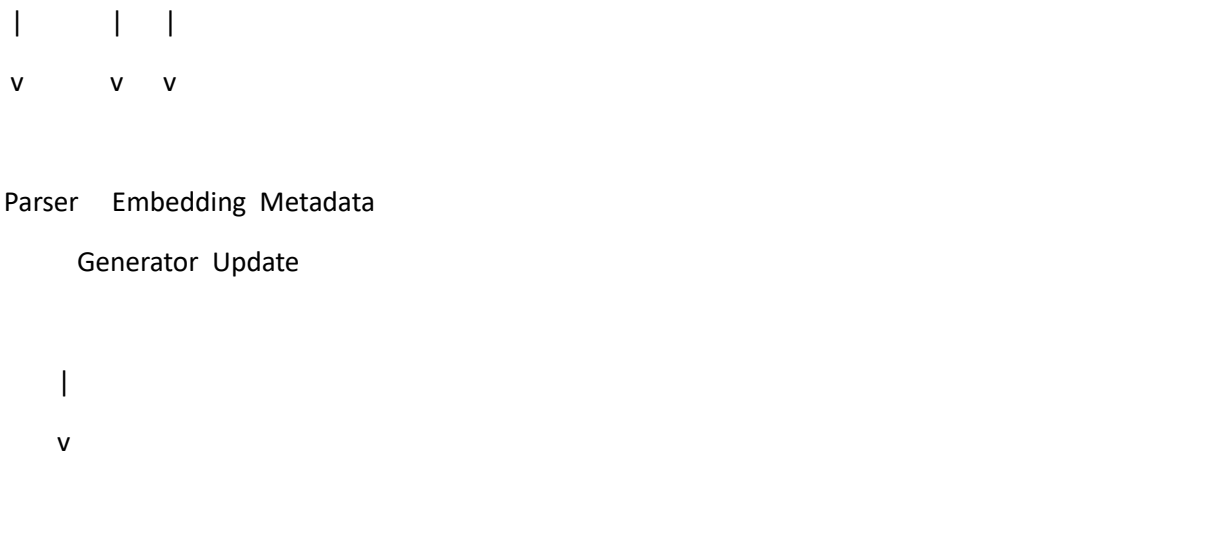
|

v

Worker Container

|

+------+------+------+




---

**Indexed content**

The worker updates:

**Text content**

- prompts
- wildcards
- notes
- documentation

---

**Metadata**

- tags
- themes
- model profiles
- copyright information
- relationships

---

**Semantic embeddings**

Generated for:

- prompts
- tags
- images

- documents

---

**User experience**

Example:

Editing:

dragon_knight.prompt

After save:

Index update scheduled

Status:

✓ File saved

⏳ Semantic index updating...

No UI freezing.

---

**Question 48 — Image Similarity Search**

You selected hybrid search and image analysis.

Should image similarity search be included?

Example:

User drops an image:

"Find similar images and prompts"

The system searches:

- generated images
- tags
- prompts
- styles

---

**A. No image similarity**

Only text/tag search.

---

**B. Basic image similarity** ⭐

Use image embeddings.

Features:

- find similar images
- link related prompts
- discover duplicates

---

**C. Advanced visual intelligence**

Everything in B plus:

- composition analysis
- pose similarity
- style similarity
- color analysis
- scene understanding
- image-to-prompt comparison

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the feature set powerful while staying focused on prompt management rather than becoming a full computer vision platform.

**Image similarity decision confirmed**

The project will include **basic image similarity search using image embeddings**.

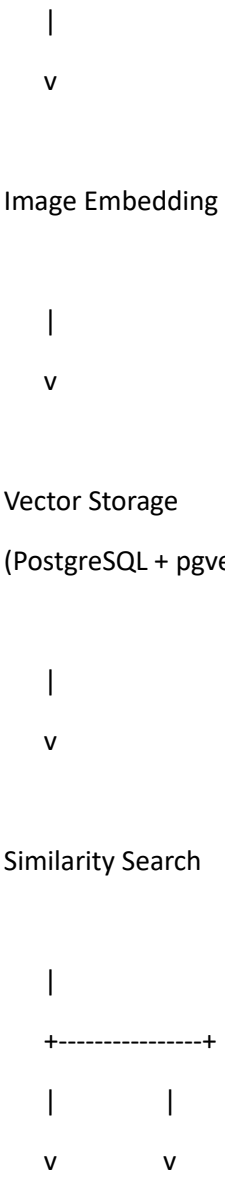
The goal:

Allow users to discover related prompts, images, and assets by visual similarity.

---

**Image Similarity Architecture**

Image Asset




---

**Supported features**

**Find similar images**

Example:

User selects:

Generated fantasy warrior image

Results:

Similar:

✓ 8 related images

✓ 5 matching prompts

✓ 3 wildcard combinations

✓ 2 model profiles

---

**Duplicate detection**

The system can identify:

- identical images
- near duplicates
- repeated generations

---

**Prompt discovery**

Image → related assets:

Image

↓

Prompt:

"Dark fantasy female knight"

Wildcard:

__fantasy_character__

Model:

Illustrious

Workflow:

Character portrait

---

**Storage**

Images themselves remain in the workspace.

Database stores:

- file reference
- metadata
- embedding vector
- relationships

---

**Question 49 — UI Layout Customization**

Since you chose a VS Code-style IDE, how customizable should the interface layout be?

**A. Fixed layout**

Simple:

- explorer
- editor
- panels

No customization.

---

**B. Panel rearrangement** ⭐

Allow:

- move panels
- resize panels
- hide/show panels
- save layouts

---

**C. Full IDE workspace customization**

Everything in B plus:

- multiple layouts
- workspace presets
- custom shortcuts
- detachable windows
- plugin-created panels

---

Which one do you want?

A, B, or C?

B

C

Excellent. This fits the overall goal: a tool that becomes useful immediately while still respecting user control.

**First launch experience decision confirmed**

The project will implement a **smart onboarding wizard**.

The application will attempt to understand the user's existing AI image workflow and prepare the environment automatically.

---

**Smart Onboarding Architecture**

First Launch

|

v

Environment Scanner

|

+----+----------+

| |

v v

System Detection User Choices

|

v

Setup Assistant

|

v

Ready Workspace

---

**Detection features**

The wizard can detect:

**ComfyUI**

Search for:

- running instances
- local API endpoints
- known installation paths
- workflow folders

Example:

Detected:

ComfyUI

✓ Running

✓ API available

✓ Port: 8188

---

**Existing assets**

Scan for:

- wildcard folders
- prompt files
- generated images
- workflows

- model metadata

Example:

Found:

Wildcards:

2,450 entries

Images:

18,300 files

Workflows:

320 files

---

**AI providers**

Setup:

- KoboldCpp endpoint detection
- API key configuration
- local model availability

---

**User control**

The wizard must always allow:

- preview before import
- choose folders
- skip steps
- modify detected settings

No automatic destructive actions.

---

**Initial workspace creation**

Example:

My_Wildcard_Studio/

├── prompts/

├── wildcards/

├── images/

├── workflows/

├── knowledge/

└── workspace.json

---

**Question 51 — Application Packaging**

The final application will run in Docker, but how should users receive it?

**A. Docker only**

Users manually install:

- Docker
- Docker Compose
- configure environment

---

**B. Docker + installation scripts** ⭐

Provide:

- one-click startup scripts
- environment checks
- automatic setup

Examples:

start_windows.bat

start_linux.sh

start_mac.sh

---

**C. Full desktop installer**

Provide:

- Windows installer
- macOS app

- Linux package

with Docker hidden internally.

---

Which one do you want?

A, B, or C?

B

Excellent. This is the best fit for the target audience: powerful enough for developers, but approachable for creators.

**Application packaging decision confirmed**

The project will use:

✅ **Docker + installation/setup scripts**

Docker remains the technical foundation, while scripts simplify deployment.

---

**Deployment Architecture**

User Machine

|

v

Setup Script

|

+------+------+

| |

v v

Docker Check Environment Setup

|

v

docker-compose.yml

|

v

Wildcard Management Studio

---

**Supported setup flows**

**Windows**

Example:

start_windows.bat

Checks:

- Docker installed
- Docker Desktop running
- available disk space
- required ports

Then:

docker compose up

---

**Linux**

Example:

./start_linux.sh

Handles:

- Docker installation checks
- permissions
- volume creation
- startup

---

**macOS**

Example:

./start_mac.sh

---

**Setup assistant**

The scripts can guide:

✓ Docker detected

✓ PostgreSQL volume created

✓ Application started

✓ Open browser:

http://localhost:xxxx

---

**Configuration files**

Example:

.env

DATABASE_URL=

REDIS_URL=

COMFYUI_URL=

DEFAULT_AI_PROVIDER=

WORKSPACE_PATH=

---

**Question 52 — Logging and Diagnostics**

A complex system like this needs diagnostics.

How should logging work?

**A. Basic logs**

Only errors and startup messages.

---

**B. Developer logs** ⭐

Include:

- API requests
- worker tasks
- import progress
- AI calls
- indexing status

---

**C. Advanced diagnostic system**

Everything in B plus:

- log viewer UI
- export diagnostics package
- performance metrics
- health dashboard

---

Which one do you want?

A, B, or C?

B

Excellent. This keeps the system professional and maintainable without turning the first version into an enterprise monitoring platform.

**Logging and diagnostics decision confirmed**

The project will implement **developer-level logging**.

The goal:

Make problems understandable during development, debugging, imports, AI operations, and Docker deployment.

---

**Logging Architecture**

Application Services

Frontend

Backend

Worker

Database Connector

AI Connectors

ComfyUI Connector

|

v

Logging Layer

|

v

Log Files / Console Output

---

**Logged components**

**Backend**

Logs:

- API requests
- database operations
- prompt processing
- wildcard expansion
- errors
- performance timings

---

**Worker**

Logs:

- import jobs
- DanTagGen processing
- embedding generation
- RAG indexing
- simulation tasks

Example:

[Worker]

Job:

Image Tagging Batch

Progress:

4500 / 10000

Status:

Running

---

**AI providers**

Logs:

- provider used
- model selected
- execution time
- errors

Sensitive data protection:

API keys are never logged.

---

**ComfyUI connector**

Logs:

- connection status
- workflow submission
- queue status

- generation results

---

**Log levels**

Support:

DEBUG

INFO

WARNING

ERROR

CRITICAL

---

**Development benefits**

Useful for:

- debugging Docker deployments
- troubleshooting imports
- validating wildcard compatibility
- improving AI workflows

---

**Question 53 — Testing Strategy**

Given the complexity of the system, what testing approach should be included?

**A. Minimal testing**

Only basic checks.

---

**B. Automated testing** ⭐

Include:

- backend unit tests
- API tests
- parser tests
- wildcard engine tests
- database tests

---

**C. Full quality pipeline**

Everything in B plus:

- frontend tests
- end-to-end browser tests
- Docker integration tests
- automated CI/CD pipeline

---

Which one do you want?

A, B, or C?

B

Excellent. This completes the overall philosophy of the project.

**Project licensing decision confirmed**

The project will be developed as an **open source project**.

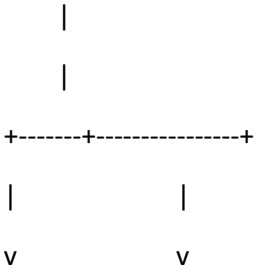
The goal:

Build a community-driven Prompt IDE that can evolve with AI image generation technologies.

---

**Open Source Architecture**

Public Repository



Workers AI Connectors

Database Analyzers

---

**Open source benefits**

**Community development**

Potential contributions:

- new model profiles
- wildcard improvements
- importers
- UI improvements
- translations
- documentation

---

**Repository structure**

Example:

wildcard-management-studio/

├── frontend/

│

├── backend/

│

├── worker/

│

├── database/

│

├── extensions/

│

├── docs/

│

├── docker/

│

├── tests/

│

└── docker-compose.yml

---

**Contribution-ready design**

The final development specification should require:

- clean architecture
- modular code
- documentation
- tests
- extension interfaces
- clear APIs

---

**We have now defined the core project vision**

The final prompt will describe:

**Wildcard Management Studio**

A Dockerized open-source Prompt IDE featuring:

- React + TypeScript interface
- Monaco editor
- FastAPI backend
- PostgreSQL + pgvector
- Redis worker system
- ComfyUI Impact Pack compatible wildcard engine
- TXT/YAML import
- .prompt native format
- AI-assisted prompt creation
- SDXL / Illustrious / NoobAI / Anima / Ideogram profiles
- KoboldCpp + cloud AI connectors
- DanTagGen/z-tipo integration
- semantic search
- RAG knowledge base

- image similarity search
- version control
- copyright analysis
- automatic backups
- workspace management
- extension architecture
- Docker Compose deployment
- open-source workflow

---

Before I generate the final **complete development prompt**, I need one last clarification:

**Question 56 — Target Audience**

Who is the main user of Wildcard Management Studio?

**A. Advanced AI image creators**

Focus:

- ComfyUI users
- prompt engineers
- LoRA/model users

---

**B. Developers building AI tools**

Focus:

- APIs
- extensions
- integrations
- coding flexibility

---

**C. Both equally** ⭐

Designed as:

- a creator tool
- a developer platform

---

Which one do you choose?

A, B, or C?

C

Excellent. This final choice completes the positioning of the project.

**Target audience decision confirmed**

Wildcard Management Studio will be designed for **both creators and developers equally**.

The project identity:

A professional open-source Prompt IDE that bridges creative AI workflows and software extensibility.

---

**Final Product Positioning**

Wildcard Management Studio

Creative Users Developers

| |

Prompt creation Extension API

Wildcards AI connectors

Model profiles Import plugins

Image organization Backend modules

AI assistance Custom tools

\ /

\ /

Professional Prompt IDE

---

**Creator-focused capabilities**

For AI artists and ComfyUI users:

- visual prompt editor
- wildcard management
- prompt generation
- model-specific optimization
- image organization
- similarity search
- prompt history
- workflow association
- AI assistance

---

**Developer-focused capabilities**

For technical users:

- modular architecture
- extension system
- API-first backend
- Docker deployment
- database-driven design
- AI provider abstraction
- custom model profiles
- import/export pipelines
- automated testing

---

**Final specification summary is now complete**

I have collected all major architecture decisions:

**Core Stack**

- Frontend: React + TypeScript
- Editor: Monaco Editor
- Backend: Python + FastAPI
- Database: PostgreSQL + pgvector

- Workers: Redis + task queue
- Deployment: Docker Compose

**Main Features**

- ComfyUI Impact Pack compatible wildcard engine
- TXT/YAML wildcard import
- .prompt native format
- YAML conversion
- AI prompt creation
- AI prompt evaluation
- model-aware prompting
- SDXL
- Illustrious
- NoobAI
- Anima
- Ideogram 4
- FLUX and other model profiles
- KoboldCpp integration
- OpenAI / Claude / Gemini support
- DanTagGen integration
- z-tipo compatibility
- semantic search
- RAG knowledge base
- image similarity search
- version history
- workspace folders
- automatic file watching
- backup system
- copyright analysis
- import ecosystem
- extension architecture
- localization framework

- smart onboarding

**Development Philosophy**

- open source
- local first
- modular
- extensible
- creator + developer focused

---

I will now prepare the final **master development prompt** that can be given directly to an AI coding agent (Claude Code, Cursor, Windsurf, Copilot, etc.) to design and implement the project.

ok, wirte it as a .md file

Data analysis can run again after your Instant limit resets. Do you want to continue without it for now?

The file-generation tool is currently unavailable because the Instant limit has been reached and the Mini model is text-only. I can't create the .md file in this response. Once the limit resets, I can generate the Markdown file directly for you.

Data analysis isn't available with this model. Try again when Instant usage resets after 4:57 PM, or upgrade to continue.